University of Patras - Polytechnic School
Department of Electrical Engineering
and Computer Technology

Department: Electronics and Computers Department

Thesis

of the student of Department of Electrical Engineering
and Computer Technology of the Polytechnic School of the University of Patras

Anastasios Zampetis of Michael

record number: 228322

Subject

Data Search & Extraction with Microservices

Supervisor

Professor Dr. Evangelos Dermatas, University of Patras

Thesis Number:

Patras, February 2025

ΠΙΣΤΟΠΟΙΗΣΗ

Πιστοποιείται ότι η διπλωματική εργασία με θέμα

Αναζήτηση & Εξόρυξη Δεδομένων με Microservices

του φοιτητή του Τμήματος Ηλεκτρολόγων Μηχανικών και Τεχνολογίας Υπολογιστών

Αναστασίου Ζαμπέτη του Μιχαήλ

(Α.Μ.: 228322)

παρουσιάστηκε δημόσια και εξετάστηκε στο τμήμα Ηλεκτρολόγων Μηχανικών και Τεχνολογίας Υπολογιστών στις

__/__/___

Ο Επιβλέπων

Δρ. Ευάγγελος Δερματάς
Καθηγητής

Ο Διευθυντής του Τομέα

Δρ. Γεώργιος Θεοδωρίδης
Αναπληρωτής Καθηγητής

CERTIFICATION

It is certified that the Thesis with Subject

Data Search & Extraction with Microservices

of the student of the Department of Electrical Engineering & Computer Technology

Anastasios Zampetis of Michael

(R.N: 228322)

Was presented publicly and defended at the Department of Electrical Engineering & Computer Technology at

__/__/____

The Supervisor

Dr. Evangelos Dermatas

The Director of the Division

Dr. George Theodoridis
Associate Professor

Thesis details

Subject: **Data Search & Extraction with Microservices**

Student: **Anastasios Zampetis of Michael**

Supervising Team
Professor Dr. Evangelos Dermatas

Thesis research period:
December 2023 - February 2025

This thesis was written in LaTeX.

Abstract

Microservice architecture is a widely adopted design approach in modern software development that emphasizes the development of specialized services with clearly defined capabilities and functions. This design approach has gained popularity with businesses attempting to increase responsiveness and reduce and simplify application development cycles. The improvements are realized by the integration of software engineering practices and IT operations that facilitates continuous integration, testing, and delivery, with minimal, if any, downtime. Microservices address the limitations of monolithic architectures, traditionally used in software development, by promoting modularity and independence of services. Every microservice is a separate process, and thus, development, deployment, and scaling can be achieved independently. The services may be developed with various programming languages and data storage mechanisms, thus enabling more flexibility and easy optimization. Although this model drastically improves scalability and maintainability, it requires effective orchestration and communication among the services. In today's world, vast volumes of data are being produced from various sources, such as Internet of Things (IoT) systems. However, this data is valuable only when properly processed, analysed and presented. To achieve this, technologies such as data caching, visualization, processing, and analytics are integrated into microservice-based systems. The use of these methods can enhance the amount of real-time data available, enabling and enhancing data driven decision-making in multiple sectors, such as energy management, smart cities, and industrial automation.

Key Words

Microservices, Monolithic design, Containers, IoT, Observability, Monitoring, Synthetic Data

Acknowledgements

To be completed...

Contents

List of Figures

List of Tables

1. Design concepts

Introduction

The design of modern software systems plays a crucial role in determining their efficiency, maintainability and scalability but also their development lifecycle. The more complex and demanding an application is, the more important choosing the right architectural approach is. This chapters explores fundamental design concepts in software architecture, with a particular focus in comparing the Monolithic and the Microservice architectures.

Monolithic architectures, traditionally the preferred approach, bundle all functions of an application into a single, unified system. While this method simplifies initial development and deployment, it often leads to challenges in flexibility and scalability as the application expands. On the other hand, the modern microservices architecture has gained popularity, due to it's enhanced modularity and scalability, thanks to decoupling each distinct functionality into separate services.

Additionally, the chapter examines application containerization, a key technology that enables microservice-based designs, by providing lightweight and fast virtualization for efficient deployment, resource utilization and scalability.

Lastly, the development of Internet of Things (IoT) applications is discussed, with a focus on device simulators and synthetic data for development and testing of production environments without the cost of setting up such systems.

By understanding these foundational concepts, we can better appreciate the design choices made in building robust, adaptable, and scalable software systems.

1.1 Monolithic Architecture

Prior to talking about Microservices and the Microservice architecture, it is important to first describe the Monolithic architecture paradigm. The Monolithic architecture approach was, until recently, the preferred design option for software development. In a Monolithic application, all the different components and functions of the business logic are combined into one indivisible system [1]. Typically, these components are the user interface, business rules, and data access functionality. While individual components might be developed separately, they remain tightly bound together [2], and any modification implemented in any of them requires the whole system to be rebuilt and redeployed [3].

This tightly bound nature creates a significant dependency problem. More often than not, development in one component requires functional changes in multiple others, adding to the development cost, complicating the build and testing process, and introducing delays in deployment. For example, a minor change in the user interface might necessitate updates to both the business logic and data access layers. Additionally, a single bug in any of the components has the potential to halt the entire application's operation and make it a nightmare for on-call engineers trying to figure out the root cause. This often results in multiple unrelated-to-the-issue teams joining in, until root cause analysis is complete, leading to wasted resources and prolonged downtime. Such situations underscore the inherent fragility of the Monolithic approach in complex systems.

Another significant drawback of Monolithic applications is their enormous codebase. Over time, as the application grows, the codebase can become cumbersome to manage and increasingly difficult to understand for new developers [2]. Implementing even minor changes may require navigating through extensive, interconnected code, leading to higher development times and increasing probability of introducing bugs. Maintenance becomes a challenge as technical debt accumulates, and the lack of modularity makes it harder to address specific areas without affecting the entire system.

Scalability is another major issue with Monolithic applications. Different components typically have conflicting resource requirements. For instance, one might be CPU-intensive while another is memory-intensive. Because of the unified design, all resource requirements must be handled together, making vertical scaling (increasing the power of a single server) impossible. Horizontal scaling (adding multiple copies of the application to distribute the load) remains the only option, but it is resource-intensive, inefficient, and often restricted due to the challenges of managing state across instances. This constraint can render Monolithic applications inappropriate for dynamic workloads or environments requiring rapid scaling.

Finally, Monolithic design allows for little to no flexibility when it comes to incorporating newer, state-of-the-art technologies. For instance, integrating a new database technology or a modern programming language might require a complete redesign and rebuild of the application, as its unified structure does not support gradual upgrades. Over time, this rigidity results in Monolithic applications becoming legacy systems. These systems often lag in performance and reliability compared to modern architectures and may eventually need to be completely redesigned and reconstructed, a process that is both time consuming and costly.

Despite these many drawbacks, Monolithic architecture is still favored for certain applications due to its core benefits. The most important one is performance. In most cases, Monolithic applications outperform their modular counterparts because all components run in the same memory space and avoid the overhead of individual services communication [2]. The simplicity of having a single executable can lead to faster runtimes and lower latency, making it suitable for scenarios where high performance is critical.

Initial design and implementation are also easier with a Monolithic approach, particularly for small to medium sized applications. Individual components are usually clearly defined at later stages, reducing the upfront complexity during the design phase. This simplicity extends to the unified build and deployment process, which simplifies configuration management, testing, and monitoring at the early stages of development.

Monolithic architecture approach is particularly well-suited for smaller applications or projects with well-defined, stable requirements. It helps to get things up and running faster, making it an ideal choice for situations where time to market is critical. Furthermore, when development complexity and deployment time come second to performance, a Monolithic application typically has the edge over a modular approach.

1.2 Microservice Architecture

Microservices and the Microservice Architecture have, in recent years, become one of the most popular design options for software applications. In the Microservice Architecture, the application is structured as a collection of independent services, called Microservices. Each Microservice corresponds to a distinct part of the business logic, executing a well-defined, unique process [1] [4]. These Microservices utilize lightweight communication mechanisms, such as API interfaces, allowing them to operate in unison and achieve the same final results as a Monolithic application but without being co-dependent. The independence of Microservices enables them to be built and tested separately, and they can be deployed and scaled independently as well. Each Microservice is designed to facilitate a single function of the application, making it focused, manageable, and easily comprehensible [5].

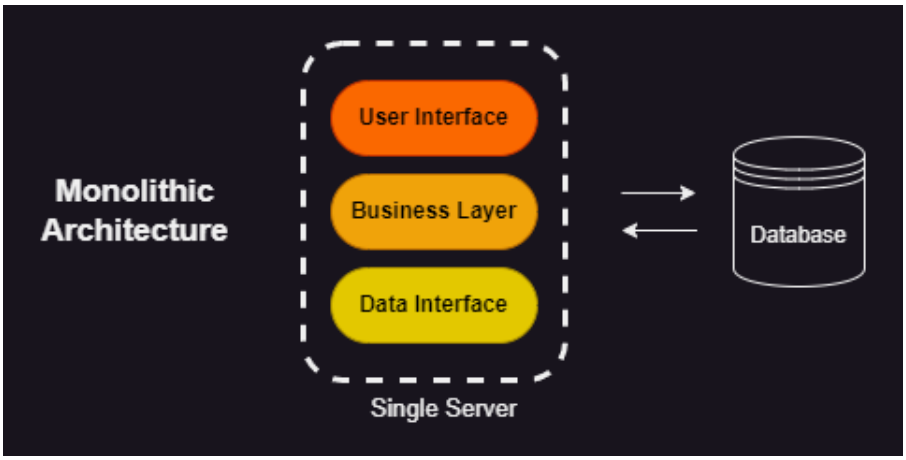

Figure 1.1: Monolithic Architecture

The advantages of this architectural approach compared to its Monolithic counterpart are significant and far-reaching. Each individual Microservice can be developed in isolation, often by different teams, without compromising or delaying the development of other parts of the application. This separation allows for parallel development, reducing bottlenecks and improving efficiency. Consequently, different components of the application can be updated and enhanced asynchronously, resulting in quicker and more frequent deployments. The build and deployment process is streamlined and more resource-efficient since only the Microservice being updated needs to be rebuild and redeployed, rather than the entire application.

Testing is another area where Microservice architecture shines. Since each Microservice operates independently, there is no need to build and test the entire application for changes made to one component. Instead, testing can be focused solely on the Microservice being modified. This leads to faster and more efficient testing cycles, allowing development teams to identify and resolve issues in a timely manner. Similarly, debugging is simplified; when an error occurs, it is relatively easy to pinpoint the Microservice responsible for the failure. The responsible team can address the issue without disrupting the operation of the entire application, making the debugging process both quicker and more effective.

While these advantages are compelling, the most critical benefit of the Microservice Architecture is scalability. In the era of cloud-native applications, where the ability to scale up or down on demand is of utmost importance and operational costs are often usage-based, Microservice-based applications significantly outclass their Monolithic counterparts. Scaling in a Microservice application is versatile, as it is possible both vertically and horizontally. The entire application can be replicated if necessary, just like a Monolithic application. However, the true strength of Microservices lies in the ability to scale individual components. For example, if one Microservice experiences a spike in demand, only that specific service can be scaled up, optimizing resource usage.

Moreover, Microservice-based applications align seamlessly with cloud infrastructure. Since Microservices can be readily instantiated as needed, there is no requirement for maintaining multiple always-on instances to handle demand spikes, as is often the case with Monolithic applications. This elasticity in scaling reduces idle resource consumption and exponentially decreases operational costs.

Despite these numerous advantages, the Microservice architecture is not without its drawbacks. The initial development of Microservice applications requires careful and time-consuming planning and design. During the early stages of development, requirements and features are often not well defined, making it challenging to design microservices-based applications effectively. Any missteps in the design can lead to significant complications later on.

Performance is another area where Microservice-based applications can fall short compared to Monolithic ones. Because Microservices rely on lightweight communication mechanisms, such as

APIs, there is an inherent overhead in communication between the components. This can result in increased latency, making Microservice applications less suitable for time-critical operations, such as load balancers or real-time risk management. For use cases requiring the lowest possible response times, Monolithic applications often have a performance edge.

Finally, Microservice architecture may not be the best choice for on-premises applications where customers are required to set up and manage everything manually. The complexity of configuring and maintaining multiple independent services can be overwhelming for end-users, especially those without extensive technical expertise. In such scenarios, a Monolithic application might be preferable due to its simplicity and ease of deployment [6].

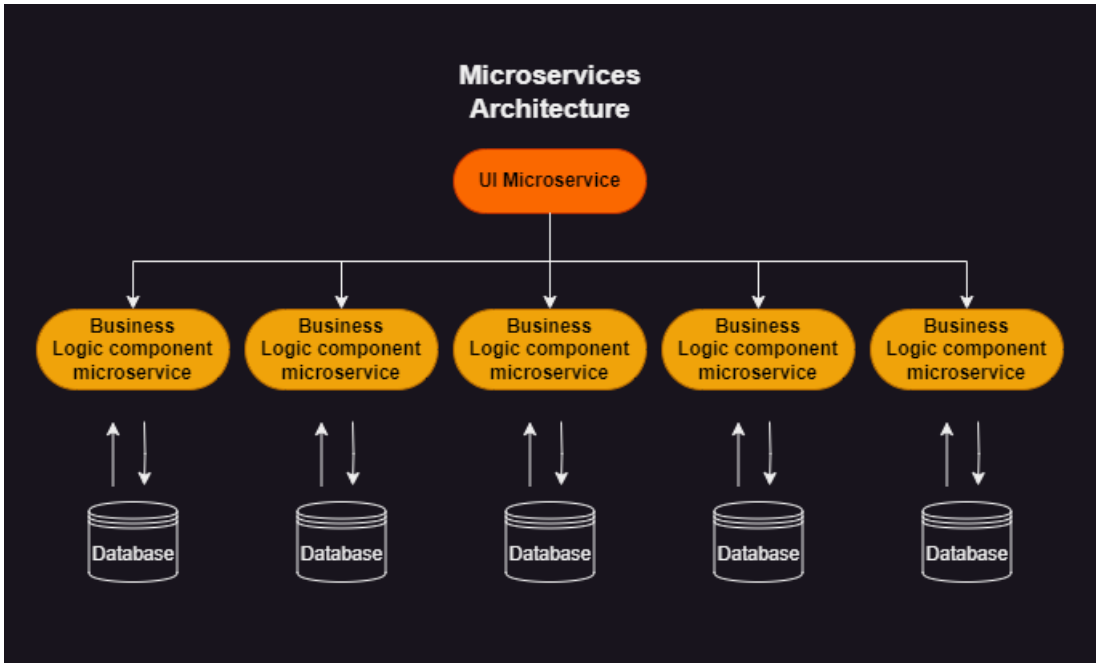

Figure 1.2: Microservice Architecture

1.3 Containers

Hand in hand with the Microservice Architecture came containers. Containers are a form of virtualization similar to virtual machines (VMs), but unlike traditional virtual machines, containers share the host system's kernel while running in isolated user spaces. This architecture makes them significantly more lightweight, efficient, and versatile compared to VMs. By eliminating the need for a full guest operating system, containers reduce resource overhead dramatically, enabling more efficient utilization of host hardware. This efficiency translates to faster startup times, reduced memory and processing power requirements, and greater performance from the same hardware infrastructure compared to VMs [7].

The lightweight nature of containers is particularly advantageous in scalable environments where applications need to scale up or down quickly in response to demand. In contrast, VMs require a separate guest operating system for each instance, which not only increases resource consumption but also lengthens deployment and startup times. Containers, on the other hand, are designed to spin up almost instantly, enabling rapid scaling and responsiveness.

Containers are created and deployed using container images, which are essentially templates. These images are built using ContainerFiles, which specify a base image and a sequence of instructions or steps to execute on top of it. Each step in the build process forms a new layer. Layering is a critical feature for image creation and deployment workflows, as it allows for reuse across different images.

For instance, if multiple container images share common steps, such as installing a specific library, those layers need to be built only once. This reuse drastically reduces build times and conserves storage space, making containerization particularly well-suited to agile development practices where frequent builds and deployments are standard.

The distribution of container images is made straightforward through container registries, platforms where images can be stored, shared, and accessed. Docker Hub is the registry associated with the most popular container platform, Docker. The ability to easily distribute container images enhances collaboration and accelerates development cycles. Teams working in different environments can just pull images from registries, ensuring consistent runtime environments and reducing potential configuration mismatches.

One of the key strengths of containers lies in their ability to provide a consistent runtime environment. By encapsulating all dependencies and configurations within the container image, containers ensure that applications run identically across development, staging, and production environments. This consistency simplifies the testing and deployment process, minimizing release time and reducing the likelihood of environment-specific bugs.

However, as the use of containers grows, managing a large number of containers across multiple hosts becomes increasingly complex. Efficient deployment and management of containerized applications require some form of orchestration. Container orchestration platforms, such as Kubernetes, have emerged as the standard solution for automating the deployment, scaling, and management of containerized applications [8]. Such platforms ensure high availability by efficiency distributing containers across nodes, maintaining desired states even during failures, and handling incident recovery automatically.

Moreover, they provide capabilities such as load balancing and resource allocation, enabling developers to build resilient and scalable systems with minimal manual intervention. Another major feature of these platforms are rolling updates, which ensure that new versions of an application can be deployed without downtime and handle rollback processes in case of failure. These features are only possible by taking advantage of the lightweight, lightning-fast deployment of containers and not only enhance the performance and reliability of applications but also reduce the operational burden on development teams.

1.4 IoT device Simulators

There's little doubt that the Internet of Things (IoT) is here to stay. IoT has reshaped the way humans and machines interact with the environment, creating smarter, more connected systems that are now strongly influencing most industries. In recent years, an increasing number of everyday devices have been equipped with various sensors and internet connectivity, enabling them to gather and transmit large volumes of data. This exponential growth of IoT devices has brought about significant opportunities, but it also presents unique challenges.

The amount of data produced by IoT devices is enormous and diverse and thus needs powerful IoT applications to process, analyze, and effectively use the data. These applications are designed to convert raw data into useful information, making devices more capable, and operations more efficient. Like any other software, though, IoT applications need to be rigorously tested prior to deployment to be reliable, secure, and function at their optimum.

One approach to testing IoT applications is to create a physical IoT network composed of the actual devices and sensors specific to the application. This setup allows for real-world data generation and provides an environment for testing the application under realistic conditions. While this method has its merits, it is not hard to notice the significant challenges and limitations it poses.

First and foremost, creating an actual IoT network for testing can be prohibitively expensive. IoT ecosystems often consist of diverse and specialized devices, many of which may be costly to acquire

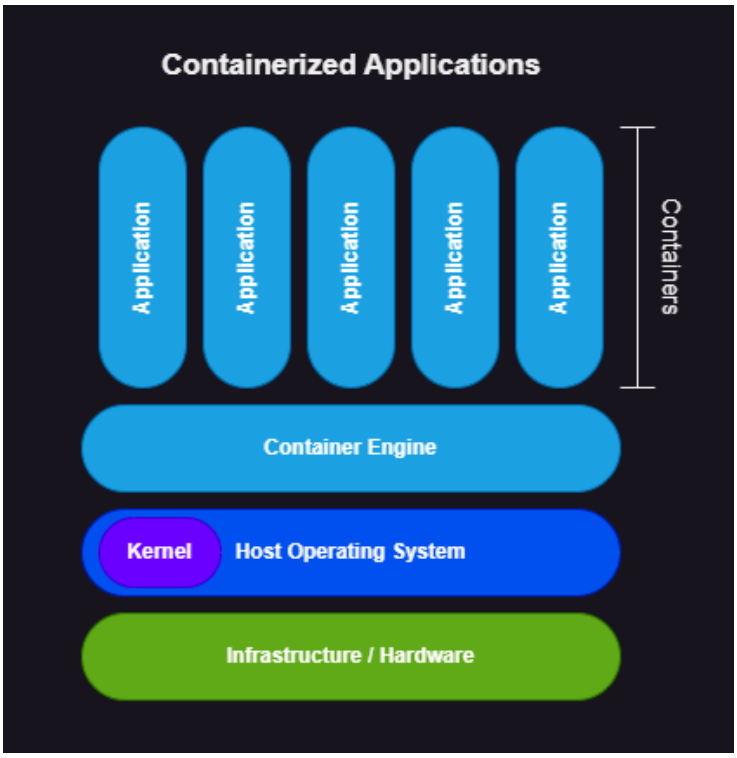

Figure 1.3: Containerized Applications

or replicate at scale. For large and complex ecosystems, the financial overhead of assembling and maintaining such a network can be quite substantial, making this approach impractical in most cases.

Furthermore, IoT applications, particularly in the early stages of development, are subject to frequent redesigns and changes. These iterations are part of the development process as requirements change, features are added, and feedback is provided. However, when an IoT network is used for testing, any significant change in the application may necessitate corresponding changes to the physical network. This adds to the development cost and introduces delays, as modifying or expanding an IoT network is both time-consuming and resource-intensive.

Even when an IoT network is already in place for deployment purposes, using it for application testing poses additional risks. Testing on a live network can expose sensitive data to potential security vulnerabilities.

To address these challenges, a more efficient and safer alternative is the use of synthetic data. Synthetic data simulates the behavior and output of real IoT devices, providing a realistic representation of IoT network activity without requiring a physical network. This approach allows developers to create virtual IoT environments tailored to specific applications, enabling thorough testing under controlled conditions.

By generating synthetic data, developers can replicate complex IoT ecosystems at a minimum cost compared with physical networks. These virtual environments can be easily modified to accommodate changes in the application, supporting agile development without the need for expensive hardware adjustments. Additionally, synthetic data eliminates the security risks associated with using real-world data during testing, as no actual devices or sensitive information are involved.

Summary

In this chapter, the fundamental application design and architecture concepts where explored. Firstly, the monolithic architecture, which despite its simplicity and performance advantages, is often difficult to scale and maintain as applications grow. In contrast, microservice architecture, is more modular,

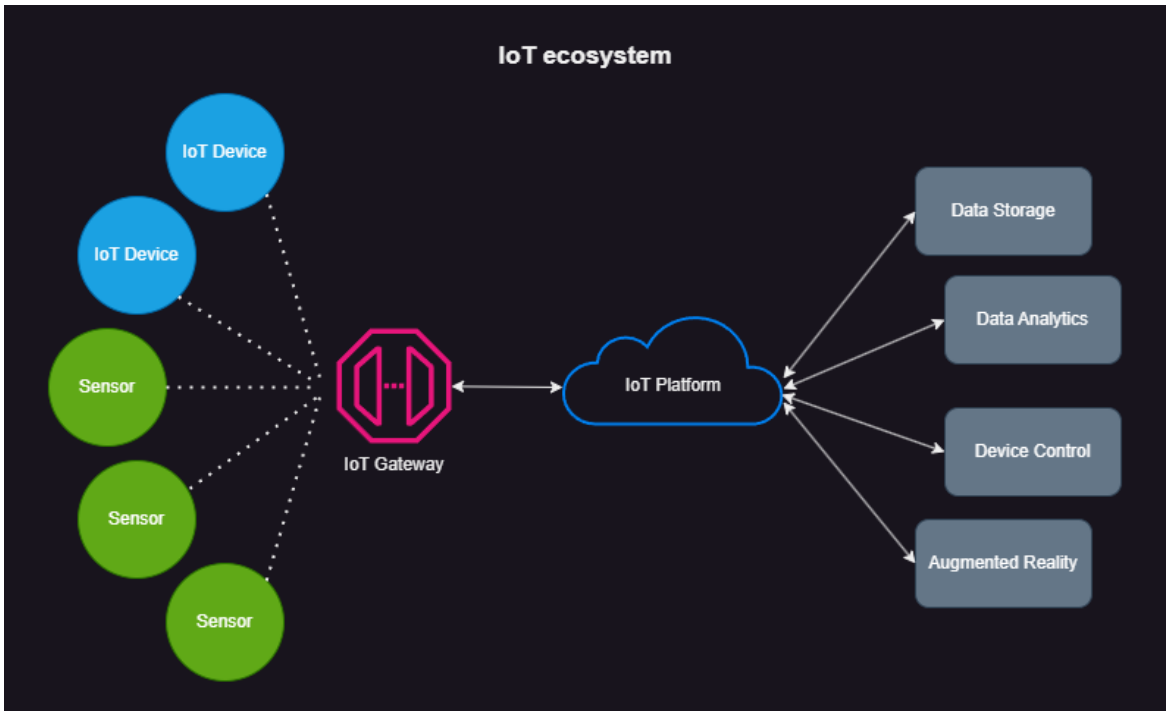

Figure 1.4: IoT ecosystem

allowing for independent deployments, flexibility and easier deployments but at the cost of increased initial complexity and communication overhead.

Containers is an essential technology, that supports the microservice architecture, enabling very lightweight and efficient deployment of services across various environments. Containers provide isolation, consistency and rapid scalability, making them a great fit for microservice based applications.

Finally, IoT device simulators were examined. Such simulators play a crucial role in testing and validating IoT systems, without requiring physical hardware, thus lowering operational costs and protecting sensitive data.

2. Technologies

Introduction

Building upon the architectural concepts presented in the previous chapter, this chapter examines the key technologies that enable the implementation of modern microservice and IoT systems. It presents the tools and platforms selected to realize the design principles discussed earlier, particularly focusing on containerization, messaging, and monitoring solutions that support distributed, scalable architectures.

The chapter explores how Docker and container technologies facilitate the microservice paradigm by providing isolated, portable deployment units. It examines MQTT as a lightweight protocol suited for IoT sensor communication, and presents monitoring tools (Prometheus, Grafana) essential for observing distributed systems. For each technology, we discuss its role in supporting the architectural goals, its key characteristics, and how it addresses specific design challenges identified in the previous chapter.

Understanding these enabling technologies and their interplay is crucial for implementing the robust, maintainable and scalable system described in the next chapter. The chapter highlights how each tool contributes to realizing core design principles like service isolation, loose coupling, and system observability.

2.1 Docker

When discussing containers, Docker cannot be left out of the discussion. Docker is an open-source platform designed to automate the creation, deployment, scaling, and management of containerized applications. It is, by far, the most widely adopted tool for containerization. Docker allows the packaging of applications and their dependencies into lightweight and easily portable containers that can be deployed consistently across a wide range of environments. This portability ensures that applications behave identically, whether running on a developer's laptop, a staging environment, or a production server.

To provision containers, Docker uses images. Docker images are read-only templates used to create containers, similar to .iso files for virtual machines, but are more lightweight and versatile. Docker images bundle everything an application needs to run, including the operating system, application code, dependencies, libraries, and configuration metadata. The metadata often includes the entry point script, a set of commands executed when the container is instantiated.

An important feature of Docker images is their layered architecture. Each layer represents a distinct change, such as adding a file, installing a package, or modifying a configuration. This layered design allows developers to build images on top of existing ones, significantly reducing build times, image sizes, and data transfer requirements.

The runtime environment responsible for building, running, and managing containers is the Docker Engine, and it consists of three main components. The first is the Docker Daemon, a background service responsible for managing Docker objects, such as containers, images, volumes, and networks. Next is the Docker Command-Line Interface (CLI), which provides a way to interact with

Docker through terminal commands. Finally, the REST API enables programmatic access to Docker's functionalities.

Docker images are created using Dockerfiles, which act as blueprints for the image creation process. A Dockerfile contains step-by-step instructions for building an image, including the base image, commands to configure the environment, install dependencies, and metadata such as port configurations and the entrypoint script. This declarative approach ensures reproducibility, as anyone with the Dockerfile can recreate the same image, ensuring consistency across teams.

Since images are meant to be portable and used over multiple environments, remote registries to store and fetch images from are crucial. Docker Hub is a free, widely used registry provided by the wider Docker ecosystem.

Hand in hand with containers, Docker enables the creation and management of other resources, critical for smooth operation. Volumes are a mechanism for persisting data generated and used by containers. Unlike ephemeral container storage, volumes ensure data remains intact even after container deletion. Docker also creates networks, enabling container inter-connectivity and communication between containers and the outside world.

For handling deployments of multiple containers in a programmatic way, Docker Compose can be utilized. Docker Compose is a tool that utilizes simple YAML files to manage multi-container deployments or applications by defining services, networks, and volumes required. This approach reduces complexity and enhances reproducibility, making it easier to manage applications with multiple inter-connected components.

Finally, Docker provides a native container orchestration platform, Docker Swarm, but on a production level, it is outclassed by other, more robust and feature-rich solutions, such as bare-metal Kubernetes or cloud Kubernetes services like Amazon Elastic Kubernetes Service (EKS) and Google Kubernetes Engine (GKE) that offer seamless integration and scalability. [9]

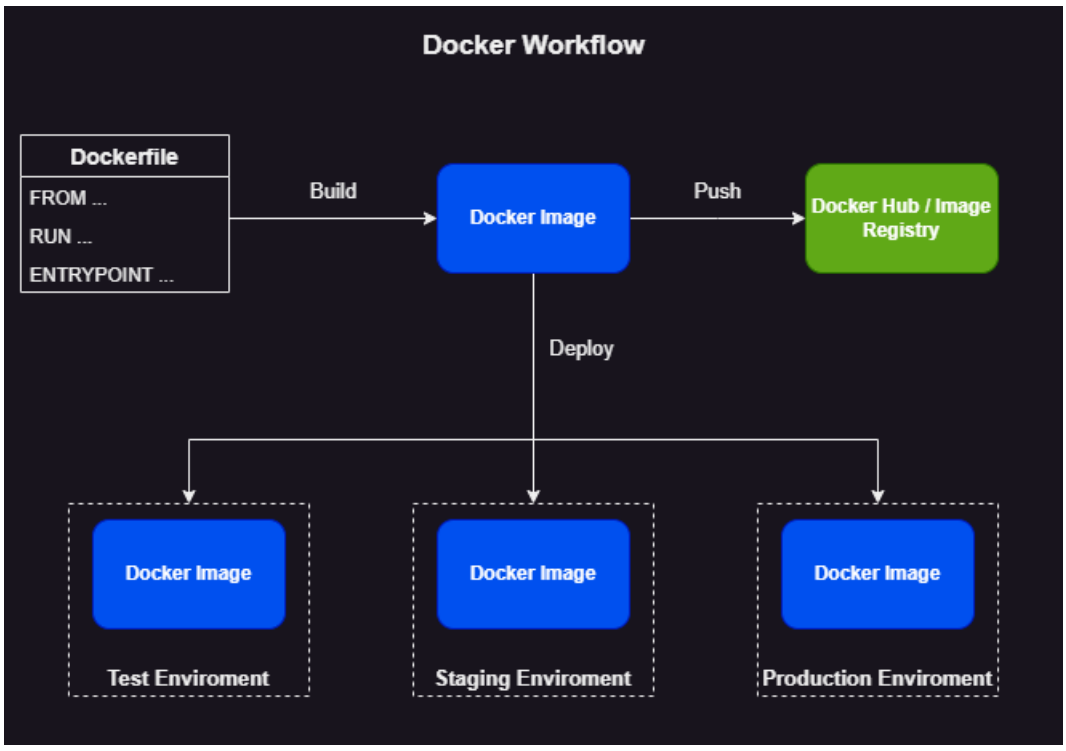

Figure 2.1: Docker Workflow

2.2 Prometheus

Prometheus is an open-source monitoring and alerting toolkit designed to provide flexible, reliable, and robust monitoring for any type of numerical data. It is considered a foundational tool for observability and a key component of modern monitoring stacks. Prometheus excels in collecting, storing, and querying time-series metrics, making it an ideal choice for monitoring infrastructure, applications, and containerized environments.

Prometheus collects and stores metrics as time-series data. Every metric is stored alongside a timestamp, allowing users to track and analyze changes over time. This feature is invaluable for identifying trends, diagnosing performance bottlenecks, and understanding long-term system behavior. Prometheus operates using a pull-based model, periodically scraping selected targets for metrics. This model ensures efficiency by fetching only the required data and enhances security by not requiring external systems to push data directly into Prometheus.

Metrics from various sources are exposed through services called Exporters, which transform raw data into a Prometheus-readable format. Prometheus includes a large ecosystem of pre-built exporters for common targets such as hardware systems, databases, and cloud platforms. Additionally, creating custom exporters is straightforward, making Prometheus highly adaptable to diverse use cases.

For retrieving, filtering, and manipulating time-series data, Prometheus Query Language (PromQL) is used. PromQL allows users to extract meaningful insights, create advanced visualizations, and define custom alerting rules. Prometheus also integrates seamlessly with Alertmanager, a companion tool for managing alerts. Users can define alerting rules based on PromQL expressions to trigger notifications when specific conditions, such as threshold breaches or anomalies, are met. Alertmanager routes these alerts to appropriate channels, such as email, Slack, or webhooks.

For short-lived jobs that may terminate before Prometheus can scrape their metrics, the Push-Gateway provides a solution. This intermediary gateway allows these jobs to push metrics at creation time, ensuring no data loss. Prometheus also supports Service Discovery, enabling automatic detection of scrape targets in dynamic environments like Kubernetes. This eliminates the need for manual configuration, making it ideal for large-scale, constantly changing systems.

Prometheus employs a highly optimized, custom-built database for storing time-series data. This database supports fine-grained retention policies, allowing users to define which metrics to retain and for how long, thereby optimizing storage usage. Prometheus further enhances usability with its ability to label metrics using key-value pairs. These labels provide additional context and facilitate filtering and aggregation, enabling multidimensional analysis of metrics.

Although Prometheus offers basic visualization capabilities, it integrates natively with Grafana, the industry-leading visualization platform. This integration allows users to build interactive, feature-rich dashboards. While Prometheus is optimized for metrics collection, it is not suitable for other types of data, such as logs. To fill these voids from Prometheus, tools like Loki are often used to handle log data, creating a comprehensive observability stack. [10, 11]

2.3 Grafana

Usually, when utilizing Prometheus to gather metrics, Grafana is employed as the visualization tool of choice. Grafana is an open-source platform for monitoring and observability, designed to enable users to visualize, query, and analyze data from an extensive range of data sources. By transforming raw metrics into actionable insights, Grafana facilitates the creation of complex, interactive dashboards, making it an essential component of most modern monitoring and observability implementations.

These dashboards are highly customizable, combining a diverse variety of visualizations, such as line graphs, heatmaps, gauges, tables, and single-stat panels. This flexibility allows teams to represent various types of information, including historical data, real-time system statuses, and long-term trends, in a visually appealing and intuitive manner.

Grafana supports integration with a broad array of data sources, including Prometheus, Loki, PostgreSQL, MySQL, and cloud services. Its ability to query multiple sources simultaneously enables advanced, multifaceted analyses, where data from diverse systems can be combined and manipulated for deeper insights.

Grafana also has support for variable creation, which allows users to dynamically assign values from the sourced data. This capability enables the construction of dynamic dashboards with parameterized views that adapt to different environments, datasets, or conditions without requiring additional customization. Such dynamic behavior unlocks templating, where reusable dashboards can be developed, significantly enhancing efficiency and consistency across monitoring setups.

Grafana excels at centralizing observability, providing a unified view of the health and performance of systems, services, and applications. Its real-time visualizations empower teams to detect and address issues as they emerge, enabling faster response times and minimizing downtime.

At the same time, the ability to chart and analyze historical data enables the identification of recurring trends and the prediction of potential problems, thus allowing the implementation of proactive measures to mitigate risks.

Lastly, Grafana offers alerting capabilities, same as Prometheus with Alertmanager, that can push notifications on various channels when a set of conditions is met [12, 13].

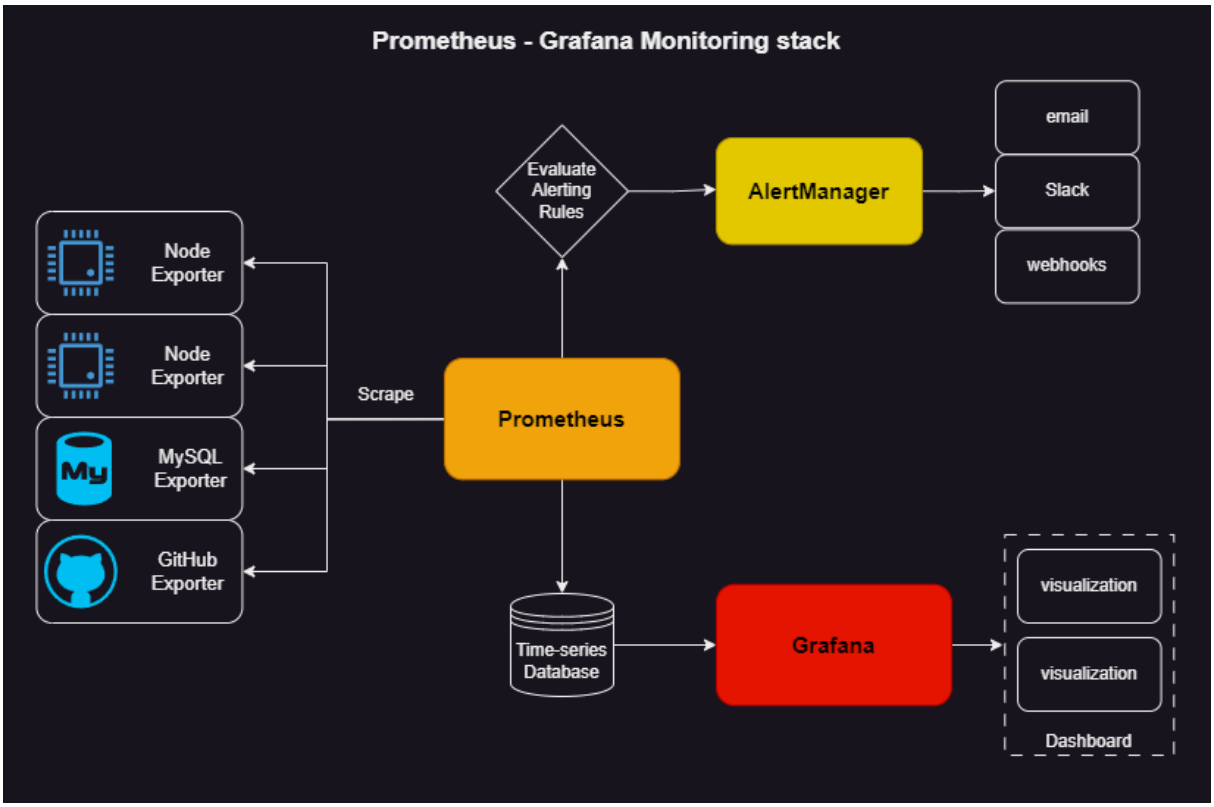

Figure 2.2: Prometheus-Grafana Monitoring stack

2.4 Message Queuing Telemetry Transport (MQTT)

Message Queuing Telemetry Transport (MQTT) is a lightweight, publish/subscribe model messaging protocol specifically designed for devices with limited computational resources and unreliable, low-bandwidth networks. Its efficiency and reliability make it a standard choice for IoT device communication and similar constrained environments.

At its core, MQTT operates using the publish/subscribe model, a decoupled communication architecture where publishers (senders) and subscribers (receivers) exchange messages through a central

broker. The MQTT broker serves as the backend system responsible for managing message coordination between clients. Its key responsibilities include receiving and filtering messages, identifying clients subscribed to specific topics, and forwarding messages to those subscribers. Popular MQTT brokers are EMQX, HiveMQ and Mosquitto. Mosquitto was preferred in this implementation for it's lightweight nature, which fits well in a microservice approach.

In typical communication, MQTT clients (which can act as publishers, subscribers, or both) establish a connection with the broker using an MQTT connection. The broker confirms the connection, ensuring both entities are ready to exchange messages. MQTT requires a TCP/IP stack on both clients and brokers for communication, and clients never connect directly with each other but only with the broker. Once connected, the client can either publish messages, subscribe to specific messages, or do both. The broker filters incoming messages using topics, which are structured hierarchically, similar to folder directories in a filesystem. The broker only sends messages to subscribers from the topics they have explicitly subscribed to.

The MQTT protocol is widely regarded as a standard for IoT data transmission and for good reason. Firstly, it requires minimal hardware resources, so it can even be used by small battery-powered microcontrollers. MQTT control messages and MQTT message headers are quite small, reducing network overhead and ensuring efficient use of bandwidth. MQTT has build-in features like quick device reconnections and quality-of-service (QoS) levels, which ensure reliable message delivery even on the unreliable, low-bandwidth and high latency cellular networks IoT devices usually operate on. The decouple nature of MQTT, combined with the low bandwidth requirements allows it to easily handle large number of clients, making it very scalable.

Despite its many strengths, MQTT does have a few limitations. The broker acts as the central node, and subsequently as a single point of failure, so any disruption or failure of the broker results in a complete communication breakdown. This risk can be mitigated through high-availability setups. Also the maximum payload is 256MB and large payloads can impact performance, however in an IoT scenario such large payloads are uncommon. Lastly, MQTT lacks native encryption but modern authentication protocols such as OAuth and TLS can be easily integrated, to enhance security [14].

Summary

This chapter explored the technological foundations that enable the microservice and IoT architecture outlined in the previous chapter. Docker provides the containerization backbone needed for service isolation and portability. MQTT offers the lightweight, publish-subscribe messaging essential for sensor communication. Prometheus and Grafana enable the monitoring and visualization capabilities required in distributed systems.

These technologies were selected because they are widely adopted, well maintained, and directly aligned with our architectural goals. While the implementation described in the next chapter makes specific technology choices and trade-offs, understanding these underlying platforms illuminates both the possibilities and constraints they impose on real-world microservice deployments.

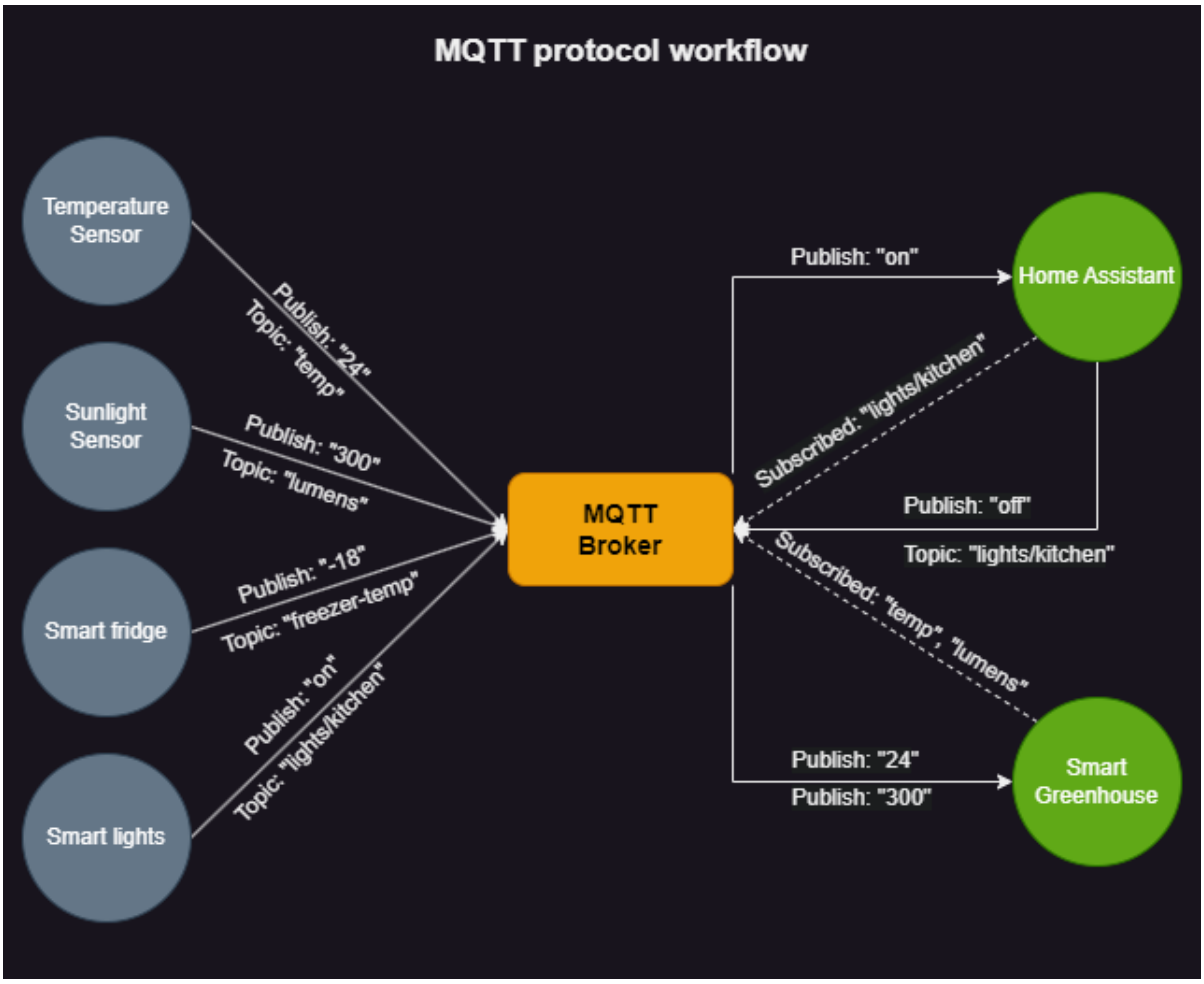

Figure 2.3: MQTT protocol workflow

3. Implementation

Introduction

This chapter describes the implementation of a sensor-driven microservice system, following the design principles and technologies outlined in previous chapters. It details the system components, their interactions and the engineering decisions taken to realize a sensor-driven microservice pipeline. The implementation focuses on: producing realistic synthetic sensor data from a re-formatted air quality dataset, delivering that data through an MQTT messaging layer, transforming and exposing metrics for Prometheus, and visualizing results with Grafana. Emphasis is placed on containerized deployment (Docker, Docker Compose) to ensure reproducibility, scalability and ease of testing in any environment.

3.1 Design and Architecture

The design of the implementation leverages a microservices architecture to simulate real-world data collection and processing. The system was designed to emulate a sensor-driven environment, utilizing containerized services for data generation, communication, transformation, and visualization.

To simulate real sensors or microcontrollers, a python application was developed. This application processes a real dataset of air quality data and extracts the data corresponding to specific timestamps. The extracted data is then published to an MQTT broker. This python application is packaged in a Docker image to enable scalability and reusability. Multiple instances of this sensor simulation image are deployed as separate Docker containers, each simulating an independent sensor.

To enable communication using the MQTT protocol between the simulated sensors and the data-processing layer, an MQTT broker was deployed. The MQTT broker, also packaged in a Docker container, receives data from the sensor containers and forwards it to the subscribed clients.

A controller service was implemented as a Python application. This application subscribes to the MQTT topics published by the sensors, processes the incoming data, and converts it into a format compatible with Prometheus. The transformed data is then exposed through an HTTP endpoint. The controller application is also containerized and deployed as a separate Docker container.

Prometheus was deployed in a Docker container and configured to periodically scrape the data exposed by the controller service and to retain it for an appropriate amount of time.

Finally, Grafana was deployed as the visualization layer of the system, again in its own Docker container. Grafana was configured to use Prometheus as its data source. Custom dashboards were created to visualize the air quality data collected from the simulated sensors, enabling real-time monitoring and analysis along with trend and spike detection.

The entire system was deployed using Docker Compose to allow for easy orchestration and management of all the containerized services and for a straightforward, scalable and reproducible deployment.

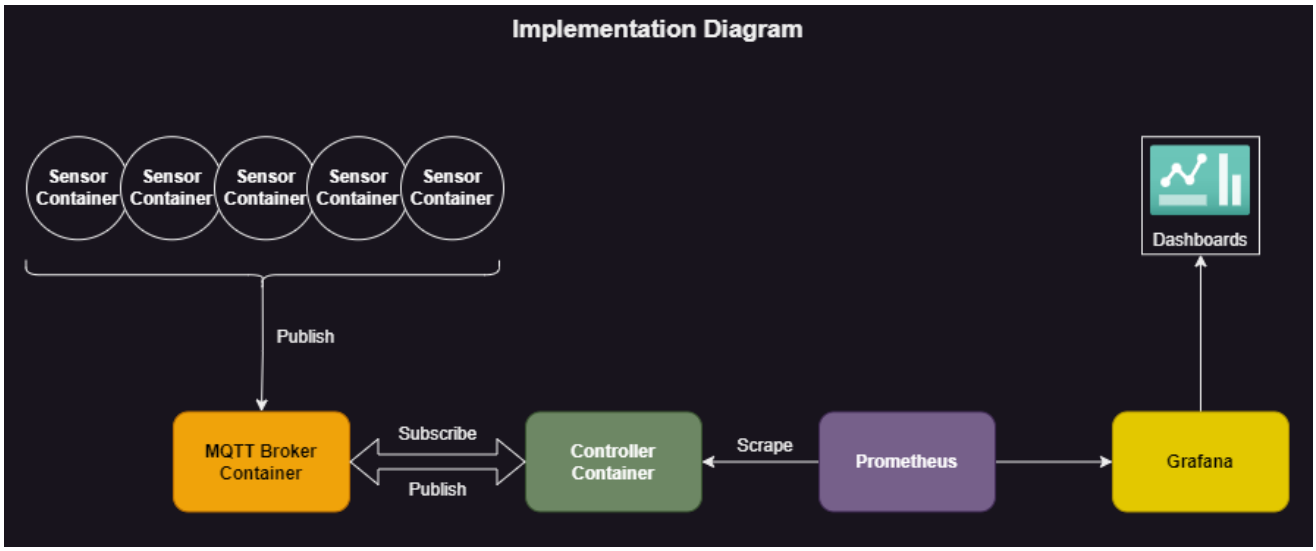

Figure 3.1: Implementation Diagram

3.2 Dataset

3.2.1 Dataset Description

The dataset used contains the responses of a gas multi-sensor device deployed on the field in an Italian city. Hourly responses averages are recorded along with gas concentrations references from a certified analyzer. The dataset contains 9357 instances of hourly averaged responses from an array of 5 metal oxide chemical sensors embedded in an Air Quality Chemical Multisensor Device. Ground Truth hourly averaged concentrations for CO, Non Metanic Hydrocarbons, Benzene, Total Nitrogen Oxides (NOx) and Nitrogen Dioxide (NO2) were provided by a co-located reference certified analyzer. Dataset can be found here.

Attribute Information	
0	Date (DD/MM/YYYY)
1	Time (HH.MM.SS)
2	True hourly averaged concentration CO in mg/m^3 (reference analyzer)
3	PT08.S1 (tin oxide) hourly averaged sensor response (nominally CO targeted)
4	True hourly averaged overall Non Metanic HydroCarbons concentration in $microg/m^3$ (reference analyzer)
5	True hourly averaged Benzene concentration in $microg/m^3$ (reference analyzer)
6	PT08.S2 (titania) hourly averaged sensor response (nominally NMHC targeted)
7	True hourly averaged NOx concentration in ppb (reference analyzer)
8	PT08.S3 (tungsten oxide) hourly averaged sensor response (nominally NOx targeted)
9	True hourly averaged NO2 concentration in $microg/m^3$ (reference analyzer)
10	PT08.S4 (tungsten oxide) hourly averaged sensor response (nominally NO2 targeted)
11	PT08.S5 (indium oxide) hourly averaged sensor response (nominally O3 targeted)
12	Temperature in °C
13	Relative Humidity (%)
14	AH Absolute Humidity

Table 3.1: Dataset Attribute Information

3.2.2 Dataset Manipulation

Dataset was originally formatted in comma-separated values (CSV) format to be easily imported in a spreadsheet. To bring the CSV formatted dataset in a programmatically friendlier json format, a python script was developed. To assist with the randomized nature of data generation by the sensor simulator, for every timestamp, a json object was created, with key an incremental integer and value the collection of metrics in json object format. Also, empty rows were removed.

```
import csv
import sys
import json
import os

# Creates a new csv file named 'edited_csv' that doesn't contain empty rows, or rows
↪  with empty fields
def csv_cleanup(csvfilename):
    with open(csvfilename, newline='') as csvfile:
        with open('edited_csv', 'w', newline='') as edited_csvfile:
            original = csv.reader(csvfile, delimiter=';')
            edited = csv.writer(edited_csvfile, delimiter=';')
            # fieldnames = next(original)
            # fieldnames.insert(0, "#")
            # edited.writerow(fieldnames)
            # i = 1
            for row in original:
                if row and any(row) and any(field.strip() for field in row):
                    # row.insert(0, i)
                    edited.writerow(row)
                    # i += 1

# Creates a json file from the edited csv file with an incremental integer as keys
def csv_to_json(filename):
    data_dict = {}
    with open('edited_csv', newline='') as csvfile:
        edited = csv.DictReader(csvfile, delimiter=';')
        key = 1
        for row in edited:
            data_dict[key] = row
            key += 1
    with open(filename+'.json', 'w') as jsonfile:
        jsonfile.write(json.dumps(data_dict, indent = 4))



csvfilename = sys.argv[1]
filename = csvfilename.split('/')[-1].split('.')[0]

csv_cleanup(csvfilename)
csv_to_json(filename)
os.remove('edited_csv')
```

The json re-formatted dataset is structured as seen below.


```
{
    "1": {
        "Date": "10/03/2004",
        "Time": "18.00.00",
        "CO(GT)": "2,6",
        "PT08.S1(CO)": "1360",
        "NMHC(GT)": "150",
        "C6H6(GT)": "11,9",
        "PT08.S2(NMHC)": "1046",
        "NOx(GT)": "166",
        "PT08.S3(NOx)": "1056",
        "NO2(GT)": "113",
        "PT08.S4(NO2)": "1692",
        "PT08.S5(O3)": "1268",
        "T": "13,6",
        "RH": "48,9",
        "AH": "0,7578",
        "": ""
    },
    "2": {
        "Date": "10/03/2004",
        "Time": "19.00.00",
        "CO(GT)": "2",
        "PT08.S1(CO)": "1292",
        "NMHC(GT)": "112",
        "C6H6(GT)": "9,4",
        "PT08.S2(NMHC)": "955",
        "NOx(GT)": "103",
        "PT08.S3(NOx)": "1174",
        "NO2(GT)": "92",
        "PT08.S4(NO2)": "1559",
        "PT08.S5(O3)": "972",
        "T": "13,3",
        "RH": "47,7",
        "AH": "0,7255",
        "": ""
    }
}
```


3.3 Sensor Simulator

3.3.1 Scenario

The sensors remain in a low-power idle state to conserve energy. They are subscribed to the ”collect-data” topic but are not actively collecting or publishing data. The controller node publishes a message to the MQTT topic ”collect-data”. This message is broadcast to all subscribed sensors by the MQTT broker. Upon receiving the trigger from the collect-data topic, sensors begin collecting air quality

metrics. After collecting the data, each sensor publishes its metrics to its respective MQTT topic and returns to an idle state.

3.3.2 Application

As described on the scenario, first action on the application is establishing a connection to the MQTT broker, making sure to reconnect in case of disconnection, which would happen often in a real case scenario utilizing an unreliable cellular connection and subscribing to the ”collect-data” topic. Then, once a message from the topic is received, the data collection and publishing script is executed.

```
import socket
import time
import subprocess
import paho.mqtt.client as mqtt_client

port = 1883  # Default port for MQTT communication
topic = "collect-data"  # Topic to subscribe to for data collection
hostname = socket.gethostname()  # Get the hostname of the machine running this
↪  script. Machine (container) hostname is enforced later-on by docker compose
client_id = 'subscribe-{}'.format(hostname)  # Unique client ID for MQTT connection
# broker = 'localhost'  # Uncomment this line for local broker testing
broker = 'host.docker.internal'  # Use when running in a Docker environment

def connect_mqtt():
    """
    Connects to the MQTT broker and sets up event handlers for connect, disconnect,
    ↪  and message events.
    """

    def on_connect(client, userdata, flags, rc):
        """
        Callback triggered upon connecting to the MQTT broker.
        """
        if rc == 0:
            print("Connected to MQTT Broker!")
            client.subscribe(topic)  # Subscribe to the specified topic
        else:
            print("Failed to connect, return code %d\n", rc)

    def on_disconnect(client, userdata, rc):
        """
        Callback triggered when the MQTT client disconnects from the broker.
        Implements an exponential backoff strategy for reconnection attempts.
        """
        FIRST_RECONNECT_DELAY = 1
        RECONNECT_RATE = 2
        MAX_RECONNECT_COUNT = 12
        MAX_RECONNECT_DELAY = 60

        print("Disconnected with result code: %s", rc)
```

```
            reconnect_count, reconnect_delay = 0, FIRST_RECONNECT_DELAY
            while reconnect_count < MAX_RECONNECT_COUNT:
                print("Reconnecting in {} seconds...".format(reconnect_delay))
                time.sleep(reconnect_delay)

                try:
                    client.reconnect()
                    print("Reconnected successfully!")
                    return
                except Exception as err:
                    print("{}. Reconnect failed. Retrying...".format(err))

                reconnect_delay *= RECONNECT_RATE
                reconnect_delay = min(reconnect_delay, MAX_RECONNECT_DELAY)
                reconnect_count += 1
            print("Reconnect failed after {} attempts.
            ↪  Exiting...".format(reconnect_count))

        def on_message(client, userdata, msg):
            """
            Callback triggered when a message is received on the subscribed topic.
            """
            print("Received `{}` from `{}` topic".format(msg.payload.decode(), msg.topic))
            subprocess.run(["./sensor_data"])  # Execute the sensor_data script upon
            ↪  message receipt

        # Create an MQTT client instance, assign the callback functions and connect
        client = mqtt_client.Client(client_id)
        client.on_connect = on_connect
        client.on_disconnect = on_disconnect
        client.on_message = on_message
        client.connect(broker, port)
        return client  # Return the configured client instance

    def run():
        """
        Main function to connect the MQTT client and start its loop.
        """
        client = connect_mqtt()
        client.loop_forever()

    if __name__ == '__main__':
        run()
```

The data collection script, again, starts by establishing a connection client with the MQTT broker. Once connection is established, the data collection process starts. To simulate a real-world scenario, to add variation between the multiple sensors and a randomization element, that also prolongs the use of the dataset before going over the same data, an elaborate process was devised. To ensure the randomized data have a real-world, logical flow, the next data point is selected from the range (previous - 2, previous + 3). This ensures that the metrics collected each time are only a few hours

apart instead of completely random, which would result in completely abnormal differences on metrics that wouldn't be observed on metrics taken a few minutes or an hour apart. It also ensures that the data point slowly moves forward in time, as to not overly repeat the same data. The starting point for this process is randomized using another script, to ensure results between sensors don't overlap heavily. Since the data collection process is ephemeral, the data points needs to be stored. This could have been achieved in a number of ways, like passing it back and forth between the main subscription script or using an environmental variable, but storing to a file was preferred as it could also be utilized if a recovery scenario was to be covered. Once the end of the database is reached, a new starting point is, again, randomly selected.

```
import json
import random, os
import socket
import subprocess
from dotenv import load_dotenv
import paho.mqtt.client as mqtt_client

port = 1883  # Default port for MQTT communication
hostname = socket.gethostname()  # Retrieve the hostname of the current machine.
↪  Hostname is enforced by docker compose.
topic = "sensor-data/{}".format(hostname)  # Define the unique topic for publishing
↪  sensor data
client_id = 'publish-{}'.format(hostname)  # Unique client ID for MQTT connection
# broker = 'localhost'  # Uncomment this line for local broker testing
broker = 'host.docker.internal'  # Use this broker when running in a Docker
↪  environment

def connect_mqtt():
    """
    Connects to the MQTT broker and sets up the on_connect callback.
    """

    def on_connect(client, userdata, flags, rc):
        """
        Callback triggered upon connecting to the MQTT broker.
        """
        if rc == 0:
            print("Connected to MQTT Broker!")
        else:
            print("Failed to connect, return code %d\n", rc)

    client = mqtt_client.Client(client_id)
    client.on_connect = on_connect
    client.connect(broker, port)
    return client

def data_gen():
    """
    Generates sensor data by selecting a random entry from a dataset.
    The selection point is controlled via an environment variable.
```

```
    """
    with open("dataset.json") as datafile:
        json_data = json.load(datafile)  # Load the dataset from the JSON file

        load_dotenv()  # Load environment variables from the .env file
        startpoint = int(os.getenv('STARTPOINT'))  # Get the STARTPOINT from the .env
        ↪ file
        # Randomize the data selection around the startpoint. Slowly moves forward in
        ↪ time, while keeping results semi-random and ensuring a logical history.
        randomizer = random.randint(startpoint - 2, startpoint + 3)
        while randomizer not in range(1, len(json_data)):  # Ensure the randomizer is
        ↪ valid
            subprocess.run(["./set_startpoint"])  # Run a script to reset the
            ↪ startpoint
            load_dotenv()  # Reload environment variables
            startpoint = int(os.getenv('STARTPOINT'))
            randomizer = random.randint(startpoint - 10, startpoint + 10)

        # Update the STARTPOINT in the .env file
        with open(".env", "w") as f:
            f.write("STARTPOINT={}".format(randomizer))

        randata = json_data[str(randomizer)]  # Fetch the random data entry
        return randata  # Return the selected data

def publish(client, data):
    """
    Publishes a message to the MQTT topic.
    """
    msg = str(data)  # Convert the data to a string
    result = client.publish(topic, msg)  # Publish the message to the topic
    status = result[0]  # Check the result status
    if status == 0:
        print("Sent `{}` to topic `{}`".format(msg, topic))
    else:
        print("Failed to send message to topic {}".format(topic))

if __name__ == '__main__':
    client = connect_mqtt()  # Establish the MQTT connection
    client.loop_start()  # Start the MQTT client loop in a separate thread
    randata = data_gen()  # Generate random sensor data
    publish(client, randata)  # Publish the generated data to the topic
    client.loop_stop()  # Stop the MQTT client loop
```

```
import json, random

def set_startpoint():
    """
    Sets a STARTPOINT value based on the dataset's size and writes it to an .env file.
```

```
    """
    with open("dataset.json") as datafile:
        json_data = json.load(datafile) # Load the dataset from a JSON file

        endpoint = round(len(json_data)/10) # Determine the upper limit for the
        ↪  STARTPOINT range
        startpoint = str(random.randint(1, endpoint))

        print("Setting STARTPOINT as {}".format(startpoint))

        with open(".env", "w") as f:
            f.write("STARTPOINT={}".format(startpoint)) # Write the STARTPOINT to the
            ↪  .env file

if __name__ == '__main__':
    set_startpoint()
```

Only libraries required, are ”paho-mqtt” and ”python-dotenv”.

3.3.3 Containerization

IoT sensors usually are embedded on microcontrollers with limited hardware resources. While minimal, optimized subsets of Python do exist, building the Python scripts and using the binaries directly, makes more sense. To stick with the minimal, lightweight enviroment expected in an IoT microcontroller, a minimal linux docker image as base is a good fit. The Alpine docker image was selected, as it is the industry standard for minimal, striped down Linux images, and current latest ones are only around 5MB uncompressed. To build the Python scripts, PyInstaller was used, but because Alpine utilizes Musl C library, instead of the GNU C library, a third party, PyInstaller-ready, Alpine-based Python image was used, that can be found here. Using this image also removes the requirement of installing PyInstaller, a great example of how docker removes environmental dependencies. The following command was used:

```
$ docker run --rm -v "${PWD}:/src" anastzampetis/pyinstaller-alpine --noconfirm
↪  --onefile --log-level DEBUG --clean <script_name>.py
```

Once the binaries were built, a simple Dockerfile was used to copy the binaries and the json formated dataset into the base Alpine image and set the binaries to be executed when running the container.

```
FROM alpine

WORKDIR /app
ADD dist/sensor_data .
ADD dist/sensor_sub .
ADD dist/set_startpoint .
ADD dataset.json .

CMD ./set_startpoint && ./sensor_sub
```

Finally to build the image, below command was executed:

```
$ docker build -t anastzampetis/sensor-emul:latest -f Dockerfile .
```

```
2024-05-30 23:26:03 thesis-app.mqtt_broker  | 1717100763: Client publish-sensor-2 closed its connection.
2024-05-30 23:26:03 thesis-app.sensor-1     | Connected to MQTT Broker!
2024-05-30 23:26:03 thesis-app.sensor-1     | Sent `{'Date': '10/07/2004', 'Time': '13.00.00', 'CO(GT)': '0,8', 'PT0
8.S1(CO)': '861', 'NMHC(GT)': '-200', 'C6H6(GT)': '6,0', 'PT08.S2(NMHC)': '811', 'NOx(GT)': '61', 'PT08.S3(NOx)': '9
97', 'NO2(GT)': '78', 'PT08.S4(NO2)': '1409', 'PT08.S5(O3)': '509', 'T': '33,3', 'RH': '19,2', 'AH': '0,9634', '': '
'}` to topic `sensor-data/sensor-1`
```

Figure 3.2: Output log of the simulated sensor

3.4 MQTT Broker

For the MQTT broker, the Eclipse Mosquitto MQTT broker was selected. Mosquitto is an open-source message broker that implements the MQTT protocol. Mosquitto is designed to be small and efficient, suitable for IoT devices with constrained resources, and fits well in a microservice enviroment. It's also very capable of handling even large-scale deployments, making it a good fit for a scalable enviroment.

To stay true to a microservice architecture and to make deployment and management easier, the Mosquitto MQTT broker was deployed in a docker container. It is already available in an official image found here. Configuring the container was simple in the scope of this implementation, since there was no need for secutiry protocols and persisting data inside the broker container because Prometheus is utilized.

```
persistence false      # No need to persist data, since it's pushed to Prometheus
listener 1883          # The listener port that clients publish to
allow_anonymous true   # Since there is no need for security, anonymous is allowed.
```

3.5 Controller Node

The controller node serves a number of important functions. Firstly, it's responsible for publishing on the "collect-data" MQTT topic, so the sensor simulators will leave the idle state and collect the data. Then, by subscribing to the topics the sensors publish to, "sensor-data/#" (usign a wildcard topic definition also allows for scaling the sensors), the controller node will receive messages from each topic, containing the sensor data.

Once these messages are received, using the "prometheus_client" Python library all metrics are converted in Prometheus format and exposed to an endpoint. Every metric is assigned to a different Gauge type Prometheus metric and labeled usign the sensor unique id. A gauge is a metric that represents a single numerical value that can arbitrarily go up or down. This part of the controller's functionality is essensially a Prometheus exporter.

Usually Prometheus exporters collect data periodically, but continiously expose last collected metrics on an HTTP endpoint. In this case, thought, to make it easier to change the rate of data collection, by changing the scraping rate of Prometheus, a different design was implemented. The controller, utilizing the "fastapi" library and "gunicorn" exposes an API endpoint. When Prometheus scrapes this endpoint, the data collection process is triggered by the controller, and after a small time delay the controller respondes with the metrics. Implementation is split between a main script and two modules.

```
import time
from fastapi import FastAPI, Response
from prometheus_client import generate_latest
from exporter_func import *

app = FastAPI(debug=False)

# Define an endpoint to serve metrics
@app.get("/metrics")
def get_metrics_app():
    start_time = time.time()

    # Connect to MQTT and start collecting data
    sub_client = connect_sub_mqtt()
    sub_client.loop_start()
    gather_data()
    time.sleep(1) # Allow time for data collection, in case sensors take time to
    ↪  respond
    sub_client.loop_stop()

    # Generate metrics data in Prometheus format
    data = generate_latest()

    end_time = time.time()
    execution_time = end_time - start_time
    print("Execution time:",execution_time)

    # Return the metrics data as a plain text HTTP response
    return Response(content=data, media_type="text/plain")

# Define a root endpoint with a simple message
@app.get("/")
async def root():
    return "Go to /metrics for gitlab metrics"

# Note for running the application locally
# Use the command:
# gunicorn -b localhost:8000 exporter:app -k uvicorn.workers.UvicornWorker
```

```
import paho.mqtt.client as mqtt_client
import json, time
from exporter_var import *

# Function to establish a connection to the MQTT broker and subscribe to a topic
def connect_sub_mqtt():

    # Callback for successful connection to the MQTT broker
    def on_connect(client, userdata, flags, rc):
        if rc == 0:
```

```
            print("Connected to MQTT Broker!")
            client.subscribe(sub_topic) # Subscribe to the specified topic
        else:
            print("Failed to connect, return code %d\n", rc)

    # Callback for handling disconnection from the MQTT broker
    def on_disconnect(client, userdata, rc):

        FIRST_RECONNECT_DELAY = 1
        RECONNECT_RATE = 2
        MAX_RECONNECT_COUNT = 12
        MAX_RECONNECT_DELAY = 60

        print("Disconnected with result code: %s", rc)
        reconnect_count, reconnect_delay = 0, FIRST_RECONNECT_DELAY
        while reconnect_count < MAX_RECONNECT_COUNT:
            print("Reconnecting in {} seconds...".format(reconnect_delay))
            time.sleep(reconnect_delay)

            try:
                client.reconnect()
                print("Reconnected successfully!")
                return
            except Exception as err:
                print("{}. Reconnect failed. Retrying...".format(err))

            reconnect_delay *= RECONNECT_RATE # Increase delay exponentially
            reconnect_delay = min(reconnect_delay, MAX_RECONNECT_DELAY)
            reconnect_count += 1
        print("Reconnect failed after {} attempts.
        ↪  Exiting...".format(reconnect_count))

    # Callback for receiving messages from the MQTT broker
    def on_message(client, userdata, msg):

        print("Received data from `{}` topic".format(msg.topic))
        promethify_data(msg)  # Process the received message for Prometheus metrics

    # Initialize the MQTT client and assign the callbacks
    client = mqtt_client.Client(sub_client_id)
    client.on_connect = on_connect
    client.on_disconnect = on_disconnect
    client.on_message = on_message
    client.connect(broker, sub_port)
    return client

# Function to continuously run the MQTT subscriber
def sub_client_run():
    sub_client = connect_sub_mqtt()
    sub_client.loop_forever()
```

```
# Function to publish a request message to collect data
def request_data(client):
    msg = 'Time to collect data.'
    result = client.publish(pub_topic, msg)
    status = result[0]
    if status == 0:
        print("Sent `{}` to topic `{}`".format(msg, pub_topic))
    else:
        print("Failed to send message to topic {}".format(pub_topic))

# Function to connect to the MQTT broker as a publisher
def connect_pub_mqtt():

    def on_connect(client, userdata, flags, rc):
        if rc == 0:
            print("Connected to MQTT Broker!")
        else:
            print("Failed to connect, return code %d\n", rc)

    client = mqtt_client.Client(pub_client_id)
    client.on_connect = on_connect
    client.connect(broker, pub_port)
    return client

# Function to gather data by publishing a request message
def gather_data():
    pub_client = connect_pub_mqtt()
    pub_client.loop_start()
    request_data(pub_client)
    pub_client.loop_stop()

# Function to assign received data to Prometheus metrics
def assign_data(prom_var, data_name, sensor, jsondata):

    value =  float(jsondata[data_name].replace(',', '.')) # Parse the value
    if  value != -200:  # Exclude no readings
        prom_var.labels(sensor).set(value) # Set the metric value with sensor label

# Function to process received MQTT messages and update Prometheus metrics
def promethify_data(msg):

    sensor = msg.topic.split('/')[1] # Extract sensor name from topic
    # Decode and parse the JSON payload
    jsondata_string = msg.payload.decode().replace("'", '"')
    jsondata = json.loads(jsondata_string)

    # Assign each metric to the corresponding Prometheus variable
    assign_data(air_quality_co_gt_gauge, "CO(GT)", sensor, jsondata)
    assign_data(air_quality_pt08s1_co_gauge, "PT08.S1(CO)", sensor, jsondata)
```

```
    assign_data(air_quality_nmhc_gt_gauge, "NMHC(GT)", sensor, jsondata)
    assign_data(air_quality_c6h6_gt_gauge, "C6H6(GT)", sensor, jsondata)
    assign_data(air_quality_pt08s2_nmhc_gauge, "PT08.S2(NMHC)", sensor, jsondata)
    assign_data(air_quality_nox_gt_gauge, "NOx(GT)", sensor, jsondata)
    assign_data(air_quality_pt08s3_nox_gauge, "PT08.S3(NOx)", sensor, jsondata)
    assign_data(air_quality_no2_gt_gauge, "NO2(GT)", sensor, jsondata)
    assign_data(air_quality_pt08s4_no2_gauge, "PT08.S4(NO2)", sensor, jsondata)
    assign_data(air_quality_pt08s5_o3_gauge, "PT08.S5(O3)", sensor, jsondata)
    assign_data(air_quality_t_gauge, "T", sensor, jsondata)
    assign_data(air_quality_rh_gauge, "RH", sensor, jsondata)
    assign_data(air_quality_ah_gauge, "AH", sensor, jsondata)
```

```
from prometheus_client import Gauge

sub_port = 1883 # Port for the subscriber
pub_port = 1883 # Port for the publisher
sub_topic = "sensor-data/#" # Subscription topic for sensor data (wildcard for all
↪  subtopics)
pub_topic = "collect-data" # Topic to publish data collection requests
sub_client_id = 'subscribe-exporter' # Client ID for the MQTT subscriber
pub_client_id = 'publish-exporter' # Client ID for the MQTT publisher
#broker = 'localhost' # uncomment for local testing
broker = 'host.docker.internal' # Docker-specific hostname for connecting to the
↪  broker


# Prometheus metrics definitions
# Each metric is defined as a Prometheus Gauge with a description and a label for
↪  'sensor'

air_quality_co_gt_gauge = Gauge('air_quality_co_gt_gauge', 'True hourly averaged
↪  concentration CO in mg/m^3 (reference analyzer)', ['sensor'])

air_quality_pt08s1_co_gauge = Gauge('air_quality_pt08s1_co_gauge', 'Tin oxide hourly
↪  averaged sensor response (nominally CO targeted)', ['sensor'])

air_quality_nmhc_gt_gauge = Gauge('air_quality_nmhc_gt_gauge', 'True hourly averaged
↪  overall Non Metanic HydroCarbons concentration in microg/m^3 (reference
↪  analyzer)', ['sensor'])

air_quality_c6h6_gt_gauge = Gauge('air_quality_c6h6_gt_gauge', 'True hourly averaged
↪  Benzene concentration in microg/m^3 (reference analyzer)', ['sensor'])

air_quality_pt08s2_nmhc_gauge = Gauge('air_quality_pt08s2_nmhc_gauge', 'Titania hourly
↪  averaged sensor response (nominally NMHC targeted)', ['sensor'])

air_quality_nox_gt_gauge = Gauge('air_quality_nox_gt_gauge', 'True hourly averaged NOx
↪  concentration in ppb (reference analyzer)', ['sensor'])
```

```
air_quality_pt08s3_nox_gauge = Gauge('air_quality_pt08s3_nox_gauge', 'Tungsten oxide
↪  hourly averaged sensor response (nominally NOx targeted)', ['sensor'])

air_quality_no2_gt_gauge = Gauge('air_quality_no2_gt_gauge', 'True hourly averaged NO2
↪  concentration in microg/m^3 (reference analyzer)', ['sensor'])

air_quality_pt08s4_no2_gauge = Gauge('air_quality_pt08s4_no2_gauge', 'Tungsten oxide
↪  hourly averaged sensor response (nominally NO2 targeted)', ['sensor'])

air_quality_pt08s5_o3_gauge = Gauge('air_quality_pt08s5_o3_gauge', 'Indium oxide
↪  hourly averaged sensor response (nominally O3 targeted)', ['sensor'])

air_quality_t_gauge = Gauge('air_quality_t_gauge', 'Temperature in C', ['sensor'])

air_quality_rh_gauge = Gauge('air_quality_rh_gauge', 'Relative Humidity (%)',
↪  ['sensor'])

air_quality_ah_gauge = Gauge('air_quality_ah_gauge', 'Absolute Humidity', ['sensor'])
```

Final output of the scripts on the endpoint that Prometheus scrapes is the Prometheus formated metrics and can be seen in an example below.

```
# HELP air_quality_co_gt_gauge True hourly averaged concentration CO in mg/m^3
↪  (reference analyzer)
# TYPE air_quality_co_gt_gauge gauge
air_quality_co_gt_gauge{sensor="sensor-5"} 4.6
air_quality_co_gt_gauge{sensor="sensor-1"} 2.7
air_quality_co_gt_gauge{sensor="sensor-2"} 2.1
air_quality_co_gt_gauge{sensor="sensor-3"} 4.3
air_quality_co_gt_gauge{sensor="sensor-4"} 1.9
# HELP air_quality_pt08s1_co_gauge Tin oxide hourly averaged sensor response
↪  (nominally CO targeted)
# TYPE air_quality_pt08s1_co_gauge gauge
air_quality_pt08s1_co_gauge{sensor="sensor-5"} 1808.0
air_quality_pt08s1_co_gauge{sensor="sensor-1"} 1280.0
air_quality_pt08s1_co_gauge{sensor="sensor-2"} 1327.0
air_quality_pt08s1_co_gauge{sensor="sensor-3"} 1559.0
air_quality_pt08s1_co_gauge{sensor="sensor-4"} 1096.0
# HELP air_quality_nmhc_gt_gauge True hourly averaged overall Non Metanic HydroCarbons
↪  concentration in microg/m^3 (reference analyzer)
# TYPE air_quality_nmhc_gt_gauge gauge
air_quality_nmhc_gt_gauge{sensor="sensor-5"} 262.0
air_quality_nmhc_gt_gauge{sensor="sensor-1"} 122.0
air_quality_nmhc_gt_gauge{sensor="sensor-2"} 256.0
air_quality_nmhc_gt_gauge{sensor="sensor-3"} 535.0
air_quality_nmhc_gt_gauge{sensor="sensor-4"} 220.0
# HELP air_quality_c6h6_gt_gauge True hourly averaged Benzene concentration in
↪  microg/m^3 (reference analyzer)
# TYPE air_quality_c6h6_gt_gauge gauge
air_quality_c6h6_gt_gauge{sensor="sensor-5"} 20.6
```

```
air_quality_c6h6_gt_gauge{sensor="sensor-1"} 9.6
air_quality_c6h6_gt_gauge{sensor="sensor-2"} 9.8
air_quality_c6h6_gt_gauge{sensor="sensor-3"} 18.9
air_quality_c6h6_gt_gauge{sensor="sensor-4"} 9.2
# HELP air_quality_pt08s2_nmhc_gauge Titania hourly averaged sensor response
↪  (nominally NMHC targeted)
# TYPE air_quality_pt08s2_nmhc_gauge gauge
air_quality_pt08s2_nmhc_gauge{sensor="sensor-5"} 1312.0
air_quality_pt08s2_nmhc_gauge{sensor="sensor-1"} 964.0
air_quality_pt08s2_nmhc_gauge{sensor="sensor-2"} 971.0
air_quality_pt08s2_nmhc_gauge{sensor="sensor-3"} 1267.0
air_quality_pt08s2_nmhc_gauge{sensor="sensor-4"} 947.0
# HELP air_quality_nox_gt_gauge True hourly averaged NOx concentration in ppb
↪  (reference analyzer)
# TYPE air_quality_nox_gt_gauge gauge
air_quality_nox_gt_gauge{sensor="sensor-5"} 261.0
air_quality_nox_gt_gauge{sensor="sensor-1"} 193.0
air_quality_nox_gt_gauge{sensor="sensor-2"} 124.0
air_quality_nox_gt_gauge{sensor="sensor-3"} 230.0
air_quality_nox_gt_gauge{sensor="sensor-4"} 115.0
# HELP air_quality_pt08s3_nox_gauge Tungsten oxide hourly averaged sensor response
↪  (nominally NOx targeted)
# TYPE air_quality_pt08s3_nox_gauge gauge
air_quality_pt08s3_nox_gauge{sensor="sensor-5"} 753.0
air_quality_pt08s3_nox_gauge{sensor="sensor-1"} 963.0
air_quality_pt08s3_nox_gauge{sensor="sensor-2"} 803.0
air_quality_pt08s3_nox_gauge{sensor="sensor-3"} 653.0
air_quality_pt08s3_nox_gauge{sensor="sensor-4"} 872.0
# HELP air_quality_no2_gt_gauge True hourly averaged NO2 concentration in microg/m^3
↪  (reference analyzer)
# TYPE air_quality_no2_gt_gauge gauge
air_quality_no2_gt_gauge{sensor="sensor-5"} 157.0
air_quality_no2_gt_gauge{sensor="sensor-1"} 113.0
air_quality_no2_gt_gauge{sensor="sensor-2"} 89.0
air_quality_no2_gt_gauge{sensor="sensor-3"} 149.0
air_quality_no2_gt_gauge{sensor="sensor-4"} 113.0
# HELP air_quality_pt08s4_no2_gauge Tungsten oxide hourly averaged sensor response
↪  (nominally NO2 targeted)
# TYPE air_quality_pt08s4_no2_gauge gauge
air_quality_pt08s4_no2_gauge{sensor="sensor-5"} 1993.0
air_quality_pt08s4_no2_gauge{sensor="sensor-1"} 1544.0
air_quality_pt08s4_no2_gauge{sensor="sensor-2"} 1705.0
air_quality_pt08s4_no2_gauge{sensor="sensor-3"} 2047.0
air_quality_pt08s4_no2_gauge{sensor="sensor-4"} 1519.0
# HELP air_quality_pt08s5_o3_gauge Indium oxide hourly averaged sensor response
↪  (nominally O3 targeted)
# TYPE air_quality_pt08s5_o3_gauge gauge
air_quality_pt08s5_o3_gauge{sensor="sensor-5"} 1698.0
air_quality_pt08s5_o3_gauge{sensor="sensor-1"} 1285.0
air_quality_pt08s5_o3_gauge{sensor="sensor-2"} 1120.0
```

```
air_quality_pt08s5_o3_gauge{sensor="sensor-3"} 1373.0
air_quality_pt08s5_o3_gauge{sensor="sensor-4"} 684.0
# HELP air_quality_t_gauge Temperature in C
# TYPE air_quality_t_gauge gauge
air_quality_t_gauge{sensor="sensor-5"} 18.4
air_quality_t_gauge{sensor="sensor-1"} 9.5
air_quality_t_gauge{sensor="sensor-2"} 21.6
air_quality_t_gauge{sensor="sensor-3"} 15.7
air_quality_t_gauge{sensor="sensor-4"} 18.6
# HELP air_quality_rh_gauge Relative Humidity (%)
# TYPE air_quality_rh_gauge gauge
air_quality_rh_gauge{sensor="sensor-5"} 41.7
air_quality_rh_gauge{sensor="sensor-1"} 64.1
air_quality_rh_gauge{sensor="sensor-2"} 41.7
air_quality_rh_gauge{sensor="sensor-3"} 62.5
air_quality_rh_gauge{sensor="sensor-4"} 36.9
# HELP air_quality_ah_gauge Absolute Humidity
# TYPE air_quality_ah_gauge gauge
air_quality_ah_gauge{sensor="sensor-5"} 0.8732
air_quality_ah_gauge{sensor="sensor-1"} 0.7597
air_quality_ah_gauge{sensor="sensor-2"} 1.0606
air_quality_ah_gauge{sensor="sensor-3"} 1.1092
air_quality_ah_gauge{sensor="sensor-4"} 0.7829
```

To package the controller node application in a docker image, a simple Dockerfile was used. Using the official Python 3.11 image as base, all python modules are copied in the image along with the requirements file in order to install dependencies. After installing the dependencies and exposing one port for communication with the MQTT broker and one for the Prometheus metrics endpoint, the application is ready to run inside the container. An entrypoint was set to run the application using Gunicorn with Uvicorn workers.

```
FROM python:3.11.0

# Set the working directory inside the container
WORKDIR /srv/exporter

# Add the main application and supporting scripts to the working directory
ADD exporter.py /srv/exporter/
ADD exporter_func.py /srv/exporter/
ADD exporter_var.py /srv/exporter/
ADD requirements.txt /srv/exporter/

# Set execute permissions for all users on the project directory
RUN chmod -R a+x /srv/exporter

# Install the Python dependencies specified in requirements.txt
RUN python3 -m pip install -r requirements.txt

# Expose the necessary ports for the Prometheus metrics endpoint and the MQTT protocol
EXPOSE 8000
```

```
EXPOSE 1883

# Set the entrypoint command to run the application using Gunicorn with Uvicorn
↪  workers
ENTRYPOINT [ "gunicorn", "-b", "0.0.0.0:8000", "exporter:app", "-k",
↪  "uvicorn.workers.UvicornWorker" ]
```

3.6 Prometheus

Setting up Prometheus is made extrememely easy, thanks to the usage of containers and the pre-build docker images available. Since Prometheus is such a widely used metrics collector and usually a core component of every observability stack, a lot of different images exist.

Usually, the difference in these images are the base images used, additional features and functionalities provided, or the time it takes for updates on source to be made available on the image. In this implementation the official image was used, prom/prometheus, which can be found here.

Using this image, only a configuration yaml file and a few deployment flags need to be provided when starting the Prometheus container. The configuration file can contain an extensive list of settings, providing substansial granularity, but in most cases, such as in this implementation, only a few settings are crucial. These settings usually are some global configurations like scrape interval or scrape timeout and the scrape configurations that are organised in jobs. Each job can have it's own settings that override the global ones and must have at least one scrape targets. Scrape targers are the endpoints that Prometheus scrapes and can either be set statically, as it is done for this case, or dynamically and targets are automatically detected using Prometheus's Service Discovery.

Prometheus graphical user interface can be access on <hosts_ip_address>:9090

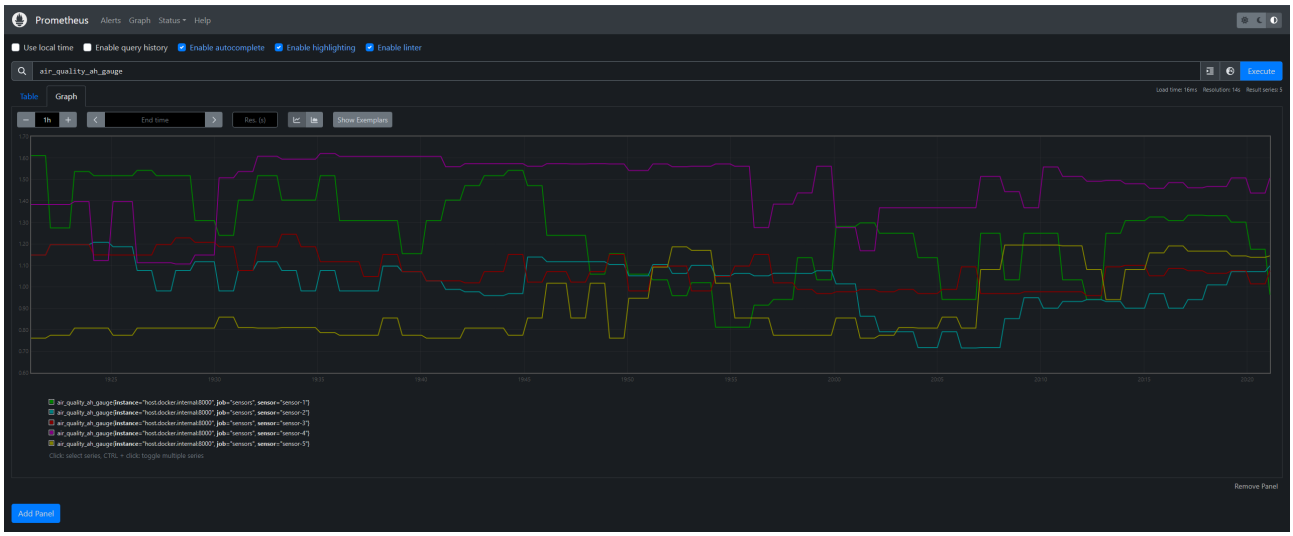

Figure 3.3: Simple graphing options on Prometheus UI

```
global:
scrape_interval:     15s
evaluation_interval: 15s

rule_files:
- prometheus_alerts_rules.yml
```

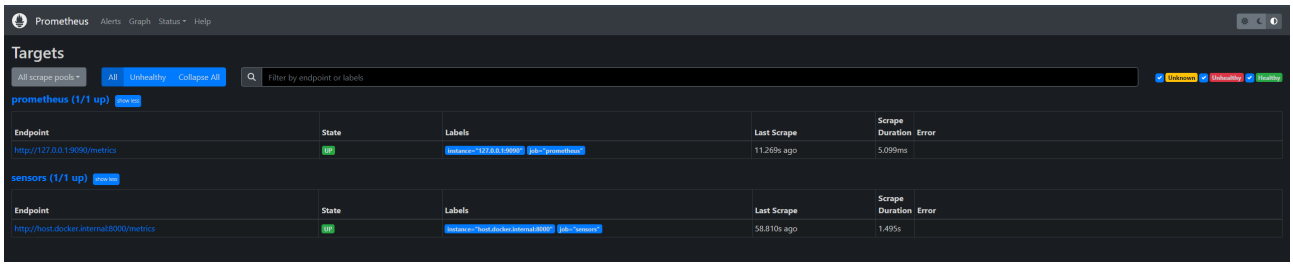

Figure 3.4: Prometheus targets UI overview

```
scrape_configs:

- job_name: prometheus # Collecting data from Prometheus itself
    static_configs:
    - targets: ['127.0.0.1:9090']

- job_name: sensors # Collecting metrics from the Controller Node
    scrape_interval: 60s
    scrape_timeout: 50s
    static_configs:
    - targets: ['host.docker.internal:8000'] # Using host.docker.internal to direct
    ↪  containerized prometheus to host's 127.0.0.1 (localhost)
```

Some very important configuration settings are passed to the Prometheus container as flags when starting it. To be able to refresh configuration without restarting the whole container, configuration yamls are mounted onto the container from local storage, to make it easy to manipulate them, and the ”–web.enable-lifecycle” flag is set. This enables users to update Prometheus configuration, for example to add additional scrape targets, by changing the config files and using Prometheus REST API to reload Prometheus. Also, to properly store the time-series database (TSDB), a host storage directory was mounted on the internal container storage directory. This ensures data integrity in case the containers crashes, or is stopped or deleted. Finally, using the ”–storage.tsdb.retention.time” flag, data retention time was set to one year.

3.7 Grafana

3.7.1 Container setup

Setting up Grafana in a container is even simpler than Prometheus, since no configuration is needed when using the https://hub.docker.com/r/grafana/grafana. Other than minor settings provided as deployment flags when running the container, all other settings are provided through the graphical user interface (GUI). Those settings can then be exported as json files to make backups of the setup and ensure easy reproducability and disaster recovery. Also very important is to mount host’s storage onto Grafana container’s data storage so enviroment persists even if container crashes or is restarted.

Grafana GUI can be access on <hosts_ip_address>:3000

3.7.2 Dashboard setup and Visualizations

Accessing Grafana using default admin credentials, allows for initial setup. If Grafana is going to be used outside of a local environment it is good practice for the default user to be either removed

altogether and role-based access control (RBAC) to be setup, or at least change default password. In this implementation, defaults where changed using deployment flags.

Prometheus was then added as a data source. Since Grafana is running inside a container, to point to host's localhost, "http://host.docker.internal:9090" was used as url. Other setting where left on default.

When monitoring metrics, such as the air quality data utilized in this implementation, it is crucial to incorporate both historical data visualization and real-time monitoring to gain a comprehensive understanding of system performance. Historical representations provide invaluable insights by revealing trends over extended periods, highlighting spikes and anomalies, and allowing for the calculation of averages and other statistical measures. These insights lead to data-driven conclusions that can inform long-term strategies and enable proactive interventions to prevent potential issues before they escalate.

Conversely, real-time monitoring offers immediate visibility into current conditions, which is essential for identifying abnormalities as they occur. This facilitates quick, reactive responses to any deviations from expected behavior and supports automated mediation processes to address issues promptly, minimizing potential negative impacts.

To address both of these needs, this implementation features two distinct dashboards. The first dashboard focuses on historical data, presenting time-series graphs of all the collected sensor metrics. This dashboard was designed as a flexible template, with the sensor parameterized as a variable to enhance organization and maintainability. A dropdown menu allows users to select specific sensors for detailed analysis, with dynamic value generation powered by PromQL to ensure seamless data retrieval and display.

The second dashboard is dedicated to providing a real-time overview of the latest collected values. It features gauge visualizations for all sensors and air quality metrics, organized by metric for easy navigation. These gauges are configured with color-coded thresholds to represent normal, slightly out-of-normal, and abnormal values, enabling users to quickly identify and address potential issues. The combination of these two dashboards ensures a holistic approach to air quality monitoring, balancing long-term analysis with immediate situational awareness.

Figure 3.5: Grafana visualization edit panel

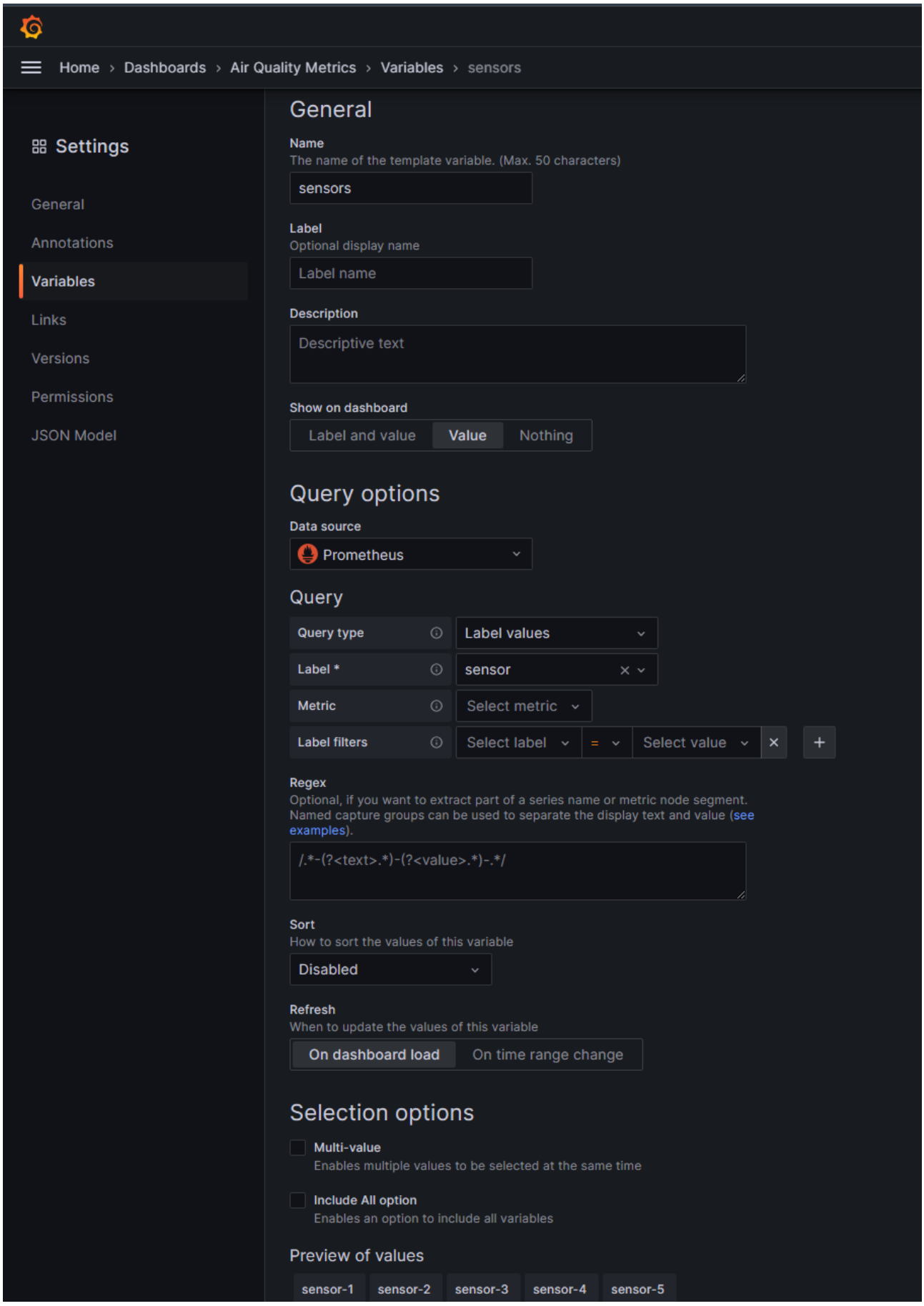

Figure 3.6: Grafana dashboard variables setup panel

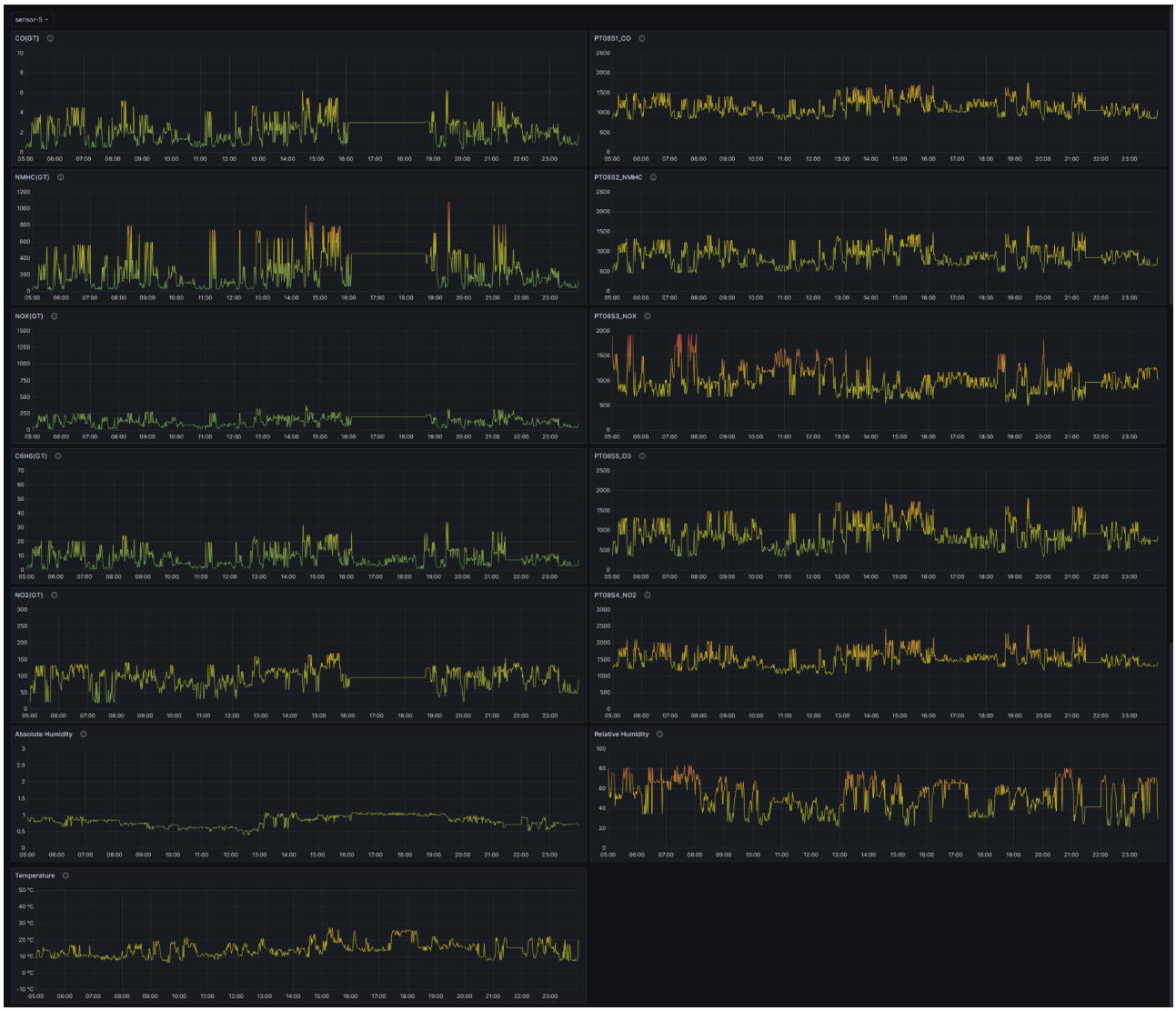

Figure 3.7: Historical data visualizations dashboard

Figure 3.8: Absolute humidity time-series visualization

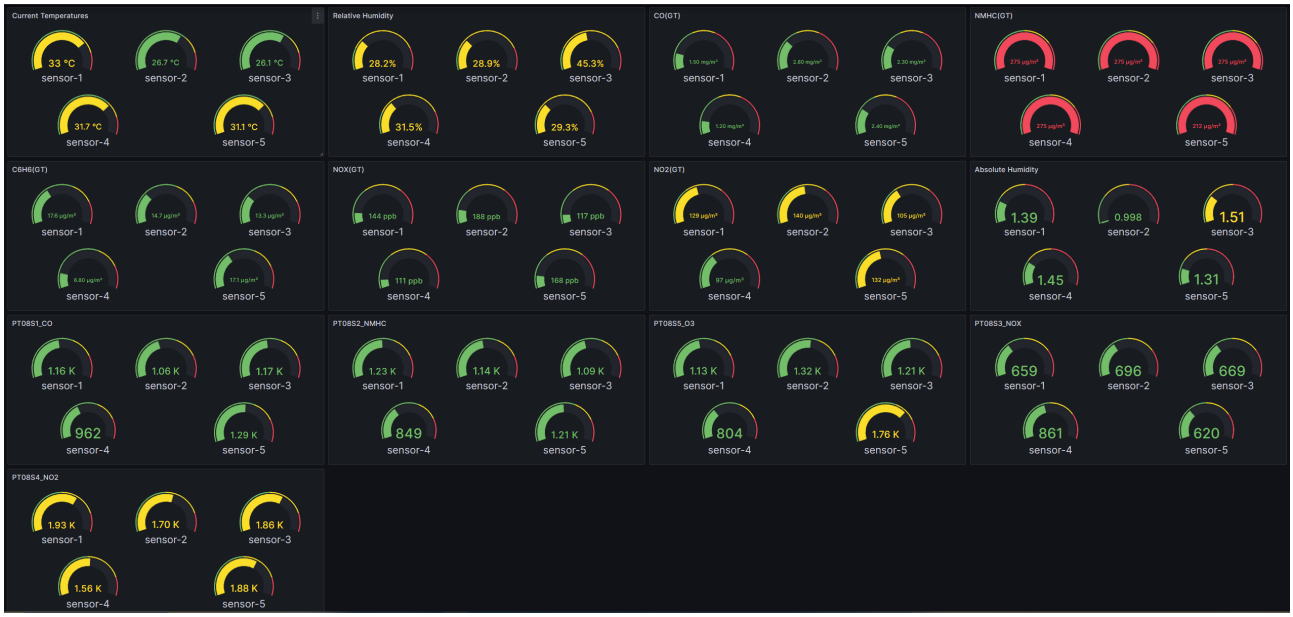

Figure 3.9: Live value gauges dashboard

Figure 3.10: Live value gauges for NMHC(GT)

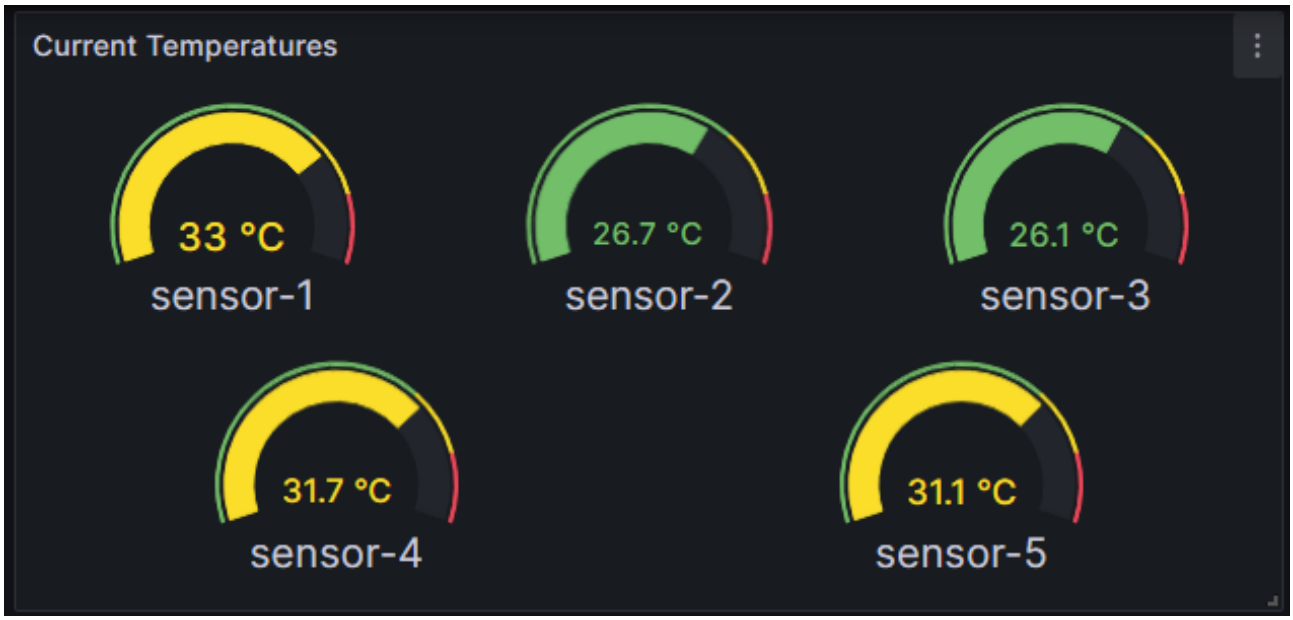

Figure 3.11: Live value gauges for temperature

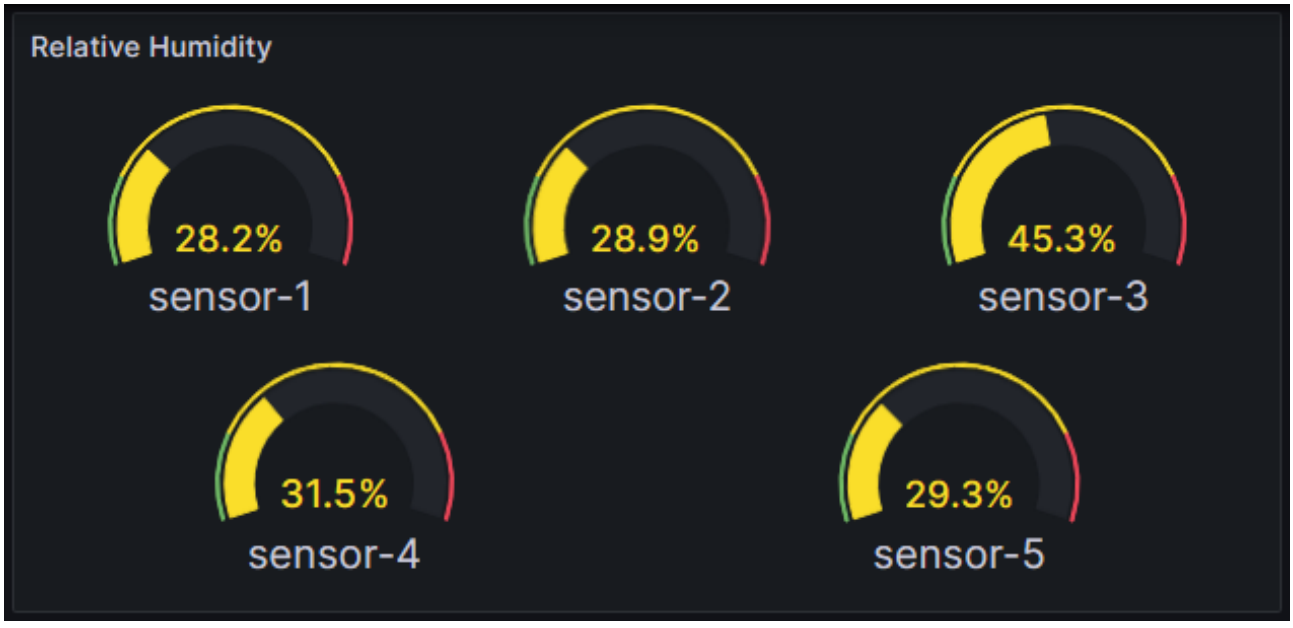

Figure 3.12: Live value gauges for relative humidity

3.8 Orchestration

To bring this implementation together the numerus containers created need to be deployed and orchestrated. For dynamic applications or for services requiring high availability (HA), an orchestration tool like Kubernetes could be utilized, but for a static implementation like this one, where a set of containers need to be deployed as a single application, Docker Compose made more sense.

Docker Compose is a tool for defining and running multi-container Docker applications using a simple YAML file. It simplifies container management by allowing defining services, networks, and volumes in a single file and spinning them up with a single command. Compared to Kubernetes, that would require setting up a cluster and defining deployments for all services/containers, Docker Compose can run on any local system equiped with Docker, making it great for local developement and small deployments.

On this implementation's Docker Compose YAML file, all components, the sensors, the MQTT broker, the controller node, Prometheus and Grafana are defined as individual services.Additionally, service settings, such as port mappings between containers and the host machine, mounted volumes, flags, environment variables and restart policies, were defined. Finally, a volume for Prometheus data was defined, to ensure data intergrity in case container was stopped and deleted.

```
services:
    mqtt_broker:
        image: "eclipse-mosquitto:2.0.18"  # Using the Eclipse Mosquitto MQTT broker
        container_name: "thesis-app.mqtt_broker"
        volumes:
        - ./mqtt_broker/mosquitto.conf:/mosquitto/config/mosquitto.conf  # Mount custom
        ↪  Mosquitto configuration
        ports:
        - "1883:1883"  # MQTT default port
        - "9001:9001"  # WebSockets support for MQTT
        restart: unless-stopped  # Automatically restart unless manually stopped

    exporter:
        image: "anastzampetis/iot-exporter"  # Custom IoT data exporter
        container_name: "thesis-app.exporter"
        ports:
        - "8000:8000"  # Expose exporter service on port 8000
        restart: unless-stopped  # Ensure the service restarts if it fails

    prometheus:
        image: "prom/prometheus:v2.46.0"  # Prometheus monitoring service
        container_name: "thesis-app.prometheus"
        volumes:
        - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml  # Custom Prometheus
        ↪  config
        -
        ↪  ./prometheus/prometheus_alerts_rules.yml:/etc/prometheus/prometheus_alerts_rules.yml
        ↪  # Alert rules (Empty but needed as file)
        - prometheus_data:/prometheus  # Persistent data storage
        ports:
        - "9090:9090"  # Expose Prometheus on port 9090
        command:
```

```
      - "--config.file=/etc/prometheus/prometheus.yml"  # Specify config file
      - "--web.enable-lifecycle"  # Allow configuration reload via API
      - "--storage.tsdb.path=/prometheus"  # Set data storage location
      - "--storage.tsdb.retention.time=1y"  # Retention policy. Keep data for 1 year
      restart: unless-stopped

  grafana:
      image: "grafana/grafana:10.2.2"  # Grafana visualization service
      container_name: "thesis-app.grafana"
      ports:
      - "3000:3000"  # Expose Grafana GUI on port 3000
      restart: unless-stopped
      environment:
      - GF_SECURITY_ADMIN_USER=admin  # Admin username for Grafana
      - GF_SECURITY_ADMIN_PASSWORD=grafana  # Admin password for Grafana
      - GF_PATHS_DATA=/var/lib/grafana  # Set data storage path
      volumes:
      - ./grafana:/var/lib/grafana  # Persist Grafana data

  # Sensor services - each sensor must be initialized separately because Docker replicas
  ↪  do not support dynamic variables and container names are not accessible from
  ↪  inside the container. (needed for MQQT client)
  sensor-1:
      image: "anastzampetis/sensor-emul"  # Custom sensor emulator image
      container_name: "thesis-app.sensor-1"
      hostname: "sensor-1"  # Set hostname for usage in the MQTT client
      restart: unless-stopped

  sensor-2:
      image: "anastzampetis/sensor-emul"
      container_name: "thesis-app.sensor-2"
      hostname: "sensor-2"
      restart: unless-stopped

  sensor-3:
      image: "anastzampetis/sensor-emul"
      container_name: "thesis-app.sensor-3"
      hostname: "sensor-3"
      restart: unless-stopped

  sensor-4:
      image: "anastzampetis/sensor-emul"
      container_name: "thesis-app.sensor-4"
      hostname: "sensor-4"
      restart: unless-stopped

  sensor-5:
      image: "anastzampetis/sensor-emul"
      container_name: "thesis-app.sensor-5"
      hostname: "sensor-5"
```

```
        restart: unless-stopped

volumes:
    prometheus_data:  # Persistent volume for Prometheus data storage
```

By using the above Docker Compose YAML and ”docker compose up” command, all required components for this implementation are reliably instantiated with the correct configurations. This approach ensures consistency and reproducibility, making deployment seamless and error-free.

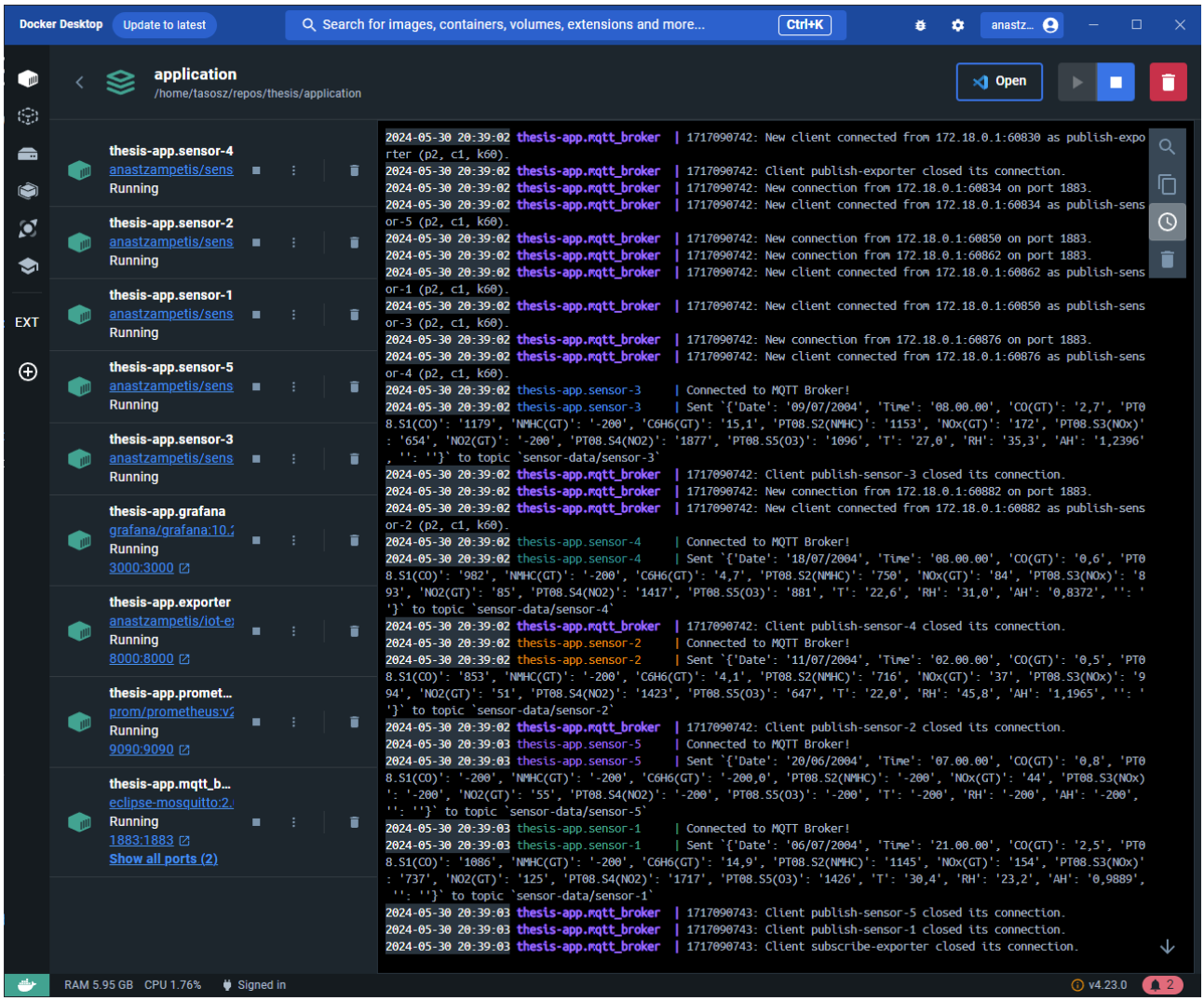

Figure 3.13: Deployed application using Docker Compose in Docker engine’s UI

Summary

This chapter presented the implemented system that realizes the designed microservice architecture: a set of containerized sensor simulators that publish synthetic but realistic air-quality readings to an MQTT broker, a controller service that ingests sensor messages, converts them to Prometheus metrics and exposes them via HTTP, Prometheus configured to scrape and retain those metrics and Grafana dashboards for visualization and monitoring. Docker Compose was used to orchestrate the services, enabling reproducible deployments and easy scaling of simulator instances. Key implementation choices included using a real-world dataset re-formatted to JSON for simulation, lightweight

Alpine-based containers to keep images small, and standard open-source tooling (Mosquitto, Prometheus, Grafana) to simplify integration and evaluation.

4. Conclusion and Future Improvements

4.1 Conclusion

This thesis proposed a microservices-based architecture to enhance data availability for real-time and historical analysis and decision-making in IoT-based environments. The fundamental aim was to showcase the benefits of microservice architecture and containerization of applications. The implementation of this thesis made apparent that it is effective, modular, and scalable to develop, maintain, manage, and deploy each separate part of an application in a microservices architecture.

One of the most crucial advantages observed was the seamless integration of all the components with the capability to scale them individually. The inherent reliability of the system also increased, as failed components can easily be reinstantiated without affecting the operation of the remainder of the services. This makes microservices a prime candidate for handling data in IoT environments, where decentralization and scalability are the prime considerations.

Furthermore, the implementation showed the effectiveness of the Prometheus and Grafana monitoring stack in providing real-time as well as historical insight into metrics. Real-time observability of system performance and data trends is critical to facilitating informed, data-driven decision-making. The implementation showcased how microservices, when correctly integrated with monitoring tools, enhance visibility in systems and result in greater operational resilience.

Lastly, the thesis highlights the necessity of proper service orchestration for handling communication overhead and facilitating seamless interaction between services. It is clearly illustrated that a well-designed microservices architecture, along with suitable containerization and monitoring techniques, can greatly enhance system performance, modularity, and resilience.

4.2 Future Improvements

The deployment of the microservices-based data search and extraction application has been successful but a improvements can be applied to multiple components to improve it's robustness, scalability and value as an observability platform.

First of all, to get real world value out of the system, simulated sensors can be swapped with real ones. A cluster of IoT sensors can be deployed around campus to monitor air quality, weather conditions, temperature reading in rooms housing critical components like servers and even detect harmful gases.

Implementing Kubernetes for container orchestration would be the next step. Utilizing Kubernetes, would allow for transforming the system towards high availability and greatly improve the scalability and robustness of the system. For better and easier resource management and scalability, a Container-as-a-Service (CaaS) solution, such as Elastic Kubernetes Service (EKS) can be used.

To ensure data integrity and to facilitate longer data retention and data storage system can be implemented. A solution with a combination of Thanos and Simple Storage Service (S3) can be utilized to not only facilitate scalable storage but also allow for multiple Prometheus instances with fast and easy data access.

To enhance the monitoring stack, alerts can be setup. Using AlertManage or Grafana's build-in support for alerts, notifications about metrics exiding typical ranges can be created. Depending on how critical these events are notifications can be anything from emails, texts to physical alarms.

Lastly, to really accentuate the ease of deploying microservice-based applications, Continuous Intergration and Continuous Deployment (CI/CD) pipelines can be developed. This would contribute to easier development and enable automated testing and seamless deployments.

Bibliography

[1] Konrad Gos and Wojciech Zabierowski. The comparison of microservice and monolithic architecture. pages 150–153, 04 2020.

[2] Ivy Wigmore Rahul Awati. What is monolithic architecture? https://www.techtarget.com/whatis/definition/monolithic-architecture, 2022.

[3] Freddy Tapia, Miguel Ángel Mora, Walter Fuertes, Hernán Aules, Edwin Flores, and Theofilos Toulkeridis. From monolithic systems to microservices: A comparative study of performance. *Applied Sciences*, 10(17), 2020.

[4] Martin Fowler James Lewis. Microservices: a definition of this new architectural term. https://martinfowler.com/articles/microservices.html, 2014.

[5] A. Chandrinos. *Upgrading Existing Allergy Symptoms Monitoring Lab Infrastructure (AllergyMap) to Containerized Environment*. Master's thesis, Department of Computer Engineering and Informatics, University of Patras, 2021.

[6] Dan Ackerson Tomas Fernandez. When microservices are a bad idea. https://semaphoreci.com/blog/bad-microservices, 2022.

[7] Claus Pahl. Containerization and the paas cloud. *IEEE Cloud Computing*, 2(3):24–31, May 2015.

[8] Dirk Merkel. Docker: lightweight linux containers for consistent development and deployment. *Linux J.*, 2014(239), mar 2014.

[9] Sriharimalapati. Containerization with docker. https://medium.com/@sriharimalapati/containerization-with-docker-6cdc2cd5889b, 2024.

[10] Yash Jani. Unified monitoring for microservices: Implementing prometheus and grafana for scalable solutions. *Journal of Artificial Intelligence, Machine Learning and Data Science*, 2(1):848–852, March 2024.

[11] Pragathi, Hrithik Maddirala, and Sneha. Implementing an effective infrastructure monitoring solution with prometheus and grafana. *Int. J. Comput. Appl.*, 186(38):7–15, September 2024.

[12] Abhishek Singh. A data visualization tool- grafana. 01 2023.

[13] Sneha M. Pragathi B.C., Hrithik Maddirala. Implementing an effective infrastructure monitoring solution with prometheus and grafana. *International Journal of Computer Applications*, 186(38):7–15, Sep 2024.

[14] Muhammad Masdani and Denny Darlis. A comprehensive study on mqtt as a low power protocol for internet of things application. *IOP Conference Series: Materials Science and Engineering*, 434:012274, 12 2018.

Code Appendix

This appendix contains the source code listings referenced throughout the implementation chapter. The code is organized by component and includes detailed comments explaining key functionality.

Dataset Processing Script

```
import csv
import sys
import json
import os

# Creates a new csv file named 'edited_csv' that doesn't contain empty rows, or rows
↪  with empty fields
def csv_cleanup(csvfilename):
    with open(csvfilename, newline='') as csvfile:
        with open('edited_csv', 'w', newline='') as edited_csvfile:
            original = csv.reader(csvfile, delimiter=';')
            edited = csv.writer(edited_csvfile, delimiter=';')
            # fieldnames = next(original)
            # fieldnames.insert(0, "#")
            # edited.writerow(fieldnames)
            # i = 1
            for row in original:
                if row and any(row) and any(field.strip() for field in row):
                    # row.insert(0, i)
                    edited.writerow(row)
                    # i += 1

# Creates a json file from the edited csv file with an incremental integer as keys
def csv_to_json(filename):
    data_dict = {}
    with open('edited_csv', newline='') as csvfile:
        edited = csv.DictReader(csvfile, delimiter=';')
        key = 1
        for row in edited:
            data_dict[key] = row
            key += 1
    with open(filename+'.json', 'w') as jsonfile:
        jsonfile.write(json.dumps(data_dict, indent = 4))
```

```
csvfilename = sys.argv[1]
filename = csvfilename.split('/')[-1].split('.')[0]

csv_cleanup(csvfilename)
csv_to_json(filename)
os.remove('edited_csv')
```

Sensor Simulator main application

```
import socket
import time
import subprocess
import paho.mqtt.client as mqtt_client

port = 1883  # Default port for MQTT communication
topic = "collect-data"  # Topic to subscribe to for data collection
hostname = socket.gethostname()  # Get the hostname of the machine running this
↪  script. Machine (container) hostname is enforced later-on by docker compose
client_id = 'subscribe-{}'.format(hostname)  # Unique client ID for MQTT connection
# broker = 'localhost'  # Uncomment this line for local broker testing
broker = 'host.docker.internal'  # Use when running in a Docker environment

def connect_mqtt():
    """
    Connects to the MQTT broker and sets up event handlers for connect, disconnect,
    ↪  and message events.
    """

    def on_connect(client, userdata, flags, rc):
        """
        Callback triggered upon connecting to the MQTT broker.
        """
        if rc == 0:
            print("Connected to MQTT Broker!")
            client.subscribe(topic)  # Subscribe to the specified topic
        else:
            print("Failed to connect, return code %d\n", rc)

    def on_disconnect(client, userdata, rc):
        """
        Callback triggered when the MQTT client disconnects from the broker.
        Implements an exponential backoff strategy for reconnection attempts.
        """
        FIRST_RECONNECT_DELAY = 1
        RECONNECT_RATE = 2
        MAX_RECONNECT_COUNT = 12
        MAX_RECONNECT_DELAY = 60
```

```
        print("Disconnected with result code: %s", rc)
        reconnect_count, reconnect_delay = 0, FIRST_RECONNECT_DELAY
        while reconnect_count < MAX_RECONNECT_COUNT:
            print("Reconnecting in {} seconds...".format(reconnect_delay))
            time.sleep(reconnect_delay)

            try:
                client.reconnect()
                print("Reconnected successfully!")
                return
            except Exception as err:
                print("{}. Reconnect failed. Retrying...".format(err))

            reconnect_delay *= RECONNECT_RATE
            reconnect_delay = min(reconnect_delay, MAX_RECONNECT_DELAY)
            reconnect_count += 1
        print("Reconnect failed after {} attempts.
        ↪  Exiting...".format(reconnect_count))

    def on_message(client, userdata, msg):
        """
        Callback triggered when a message is received on the subscribed topic.
        """
        print("Received `{}` from `{}` topic".format(msg.payload.decode(), msg.topic))
        subprocess.run(["./sensor_data"])  # Execute the sensor_data script upon
        ↪  message receipt

    # Create an MQTT client instance, assign the callback functions and connect
    client = mqtt_client.Client(client_id)
    client.on_connect = on_connect
    client.on_disconnect = on_disconnect
    client.on_message = on_message
    client.connect(broker, port)
    return client  # Return the configured client instance

def run():
    """
    Main function to connect the MQTT client and start its loop.
    """
    client = connect_mqtt()
    client.loop_forever()

if __name__ == '__main__':
    run()
```

Sensor Simulator Data Generation and Publishing script

```
import json
import random, os
import socket
import subprocess
from dotenv import load_dotenv
import paho.mqtt.client as mqtt_client

port = 1883  # Default port for MQTT communication
hostname = socket.gethostname()  # Retrieve the hostname of the current machine.
↪  Hostname is enforced by docker compose.
topic = "sensor-data/{}".format(hostname)  # Define the unique topic for publishing
↪  sensor data
client_id = 'publish-{}'.format(hostname)  # Unique client ID for MQTT connection
# broker = 'localhost'  # Uncomment this line for local broker testing
broker = 'host.docker.internal'  # Use this broker when running in a Docker
↪  environment

def connect_mqtt():
    """
    Connects to the MQTT broker and sets up the on_connect callback.
    """

    def on_connect(client, userdata, flags, rc):
        """
        Callback triggered upon connecting to the MQTT broker.
        """
        if rc == 0:
            print("Connected to MQTT Broker!")
        else:
            print("Failed to connect, return code %d\n", rc)

    client = mqtt_client.Client(client_id)
    client.on_connect = on_connect
    client.connect(broker, port)
    return client

def data_gen():
    """
    Generates sensor data by selecting a random entry from a dataset.
    The selection point is controlled via an environment variable.
    """
    with open("dataset.json") as datafile:
        json_data = json.load(datafile)  # Load the dataset from the JSON file

        load_dotenv()  # Load environment variables from the .env file
        startpoint = int(os.getenv('STARTPOINT'))  # Get the STARTPOINT from the .env
        ↪  file
        # Randomize the data selection around the startpoint. Slowly moves forward in
        ↪  time, while keeping results semi-random and ensuring a logical history.
```

```
            randomizer = random.randint(startpoint - 2, startpoint + 3)
            while randomizer not in range(1, len(json_data)):  # Ensure the randomizer is
            ↪  valid
                subprocess.run(["./set_startpoint"])  # Run a script to reset the
                ↪  startpoint
                load_dotenv()  # Reload environment variables
                startpoint = int(os.getenv('STARTPOINT'))
                randomizer = random.randint(startpoint - 10, startpoint + 10)

            # Update the STARTPOINT in the .env file
            with open(".env", "w") as f:
                f.write("STARTPOINT={}".format(randomizer))

            randata = json_data[str(randomizer)]  # Fetch the random data entry
            return randata  # Return the selected data

    def publish(client, data):
        """
        Publishes a message to the MQTT topic.
        """
        msg = str(data)  # Convert the data to a string
        result = client.publish(topic, msg)  # Publish the message to the topic
        status = result[0]  # Check the result status
        if status == 0:
            print("Sent `{}` to topic `{}`".format(msg, topic))
        else:
            print("Failed to send message to topic {}".format(topic))

    if __name__ == '__main__':
        client = connect_mqtt()  # Establish the MQTT connection
        client.loop_start()  # Start the MQTT client loop in a separate thread
        randata = data_gen()  # Generate random sensor data
        publish(client, randata)  # Publish the generated data to the topic
        client.loop_stop()  # Stop the MQTT client loop
```

Startpoint Setter Script

```
    import json, random

    def set_startpoint():
        """
        Sets a STARTPOINT value based on the dataset's size and writes it to an .env file.
        """
        with open("dataset.json") as datafile:
            json_data = json.load(datafile) # Load the dataset from a JSON file

            endpoint = round(len(json_data)/10) # Determine the upper limit for the
            ↪  STARTPOINT range
```

```
        startpoint = str(random.randint(1, endpoint))

        print("Setting STARTPOINT as {}".format(startpoint))

        with open(".env", "w") as f:
            f.write("STARTPOINT={}".format(startpoint)) # Write the STARTPOINT to the
            ↪  .env file

if __name__ == '__main__':
    set_startpoint()
```

Sensor Simulator Dockerfile

```
import json, random

def set_startpoint():
    """
    Sets a STARTPOINT value based on the dataset's size and writes it to an .env file.
    """
    with open("dataset.json") as datafile:
        json_data = json.load(datafile) # Load the dataset from a JSON file

        endpoint = round(len(json_data)/10) # Determine the upper limit for the
        ↪  STARTPOINT range
        startpoint = str(random.randint(1, endpoint))

        print("Setting STARTPOINT as {}".format(startpoint))

        with open(".env", "w") as f:
            f.write("STARTPOINT={}".format(startpoint)) # Write the STARTPOINT to the
            ↪  .env file

if __name__ == '__main__':
    set_startpoint()
```

Controller Node main application

```
import time
from fastapi import FastAPI, Response
from prometheus_client import generate_latest
from exporter_func import *

app = FastAPI(debug=False)

# Define an endpoint to serve metrics
@app.get("/metrics")
```

```
    def get_metrics_app():
        start_time = time.time()

        # Connect to MQTT and start collecting data
        sub_client = connect_sub_mqtt()
        sub_client.loop_start()
        gather_data()
        time.sleep(1) # Allow time for data collection, in case sensors take time to
        ↪  respond
        sub_client.loop_stop()

        # Generate metrics data in Prometheus format
        data = generate_latest()

        end_time = time.time()
        execution_time = end_time - start_time
        print("Execution time:",execution_time)

        # Return the metrics data as a plain text HTTP response
        return Response(content=data, media_type="text/plain")

    # Define a root endpoint with a simple message
    @app.get("/")
    async def root():
        return "Go to /metrics for gitlab metrics"

    # Note for running the application locally
    # Use the command:
    # gunicorn -b localhost:8000 exporter:app -k uvicorn.workers.UvicornWorker
```

Controller Node Data collection and Exporter Functions

```
    import paho.mqtt.client as mqtt_client
    import json, time
    from exporter_var import *

    # Function to establish a connection to the MQTT broker and subscribe to a topic
    def connect_sub_mqtt():

        # Callback for successful connection to the MQTT broker
        def on_connect(client, userdata, flags, rc):
            if rc == 0:
                print("Connected to MQTT Broker!")
                client.subscribe(sub_topic) # Subscribe to the specified topic
            else:
                print("Failed to connect, return code %d\n", rc)

        # Callback for handling disconnection from the MQTT broker
```

```
    def on_disconnect(client, userdata, rc):

        FIRST_RECONNECT_DELAY = 1
        RECONNECT_RATE = 2
        MAX_RECONNECT_COUNT = 12
        MAX_RECONNECT_DELAY = 60

        print("Disconnected with result code: %s", rc)
        reconnect_count, reconnect_delay = 0, FIRST_RECONNECT_DELAY
        while reconnect_count < MAX_RECONNECT_COUNT:
            print("Reconnecting in {} seconds...".format(reconnect_delay))
            time.sleep(reconnect_delay)

            try:
                client.reconnect()
                print("Reconnected successfully!")
                return
            except Exception as err:
                print("{}. Reconnect failed. Retrying...".format(err))

            reconnect_delay *= RECONNECT_RATE # Increase delay exponentially
            reconnect_delay = min(reconnect_delay, MAX_RECONNECT_DELAY)
            reconnect_count += 1
        print("Reconnect failed after {} attempts.
        ↪  Exiting...".format(reconnect_count))

    # Callback for receiving messages from the MQTT broker
    def on_message(client, userdata, msg):

        print("Received data from `{}` topic".format(msg.topic))
        promethify_data(msg)  # Process the received message for Prometheus metrics

    # Initialize the MQTT client and assign the callbacks
    client = mqtt_client.Client(sub_client_id)
    client.on_connect = on_connect
    client.on_disconnect = on_disconnect
    client.on_message = on_message
    client.connect(broker, sub_port)
    return client

# Function to continuously run the MQTT subscriber
def sub_client_run():
    sub_client = connect_sub_mqtt()
    sub_client.loop_forever()

# Function to publish a request message to collect data
def request_data(client):
    msg = 'Time to collect data.'
    result = client.publish(pub_topic, msg)
    status = result[0]
```

```
        if status == 0:
            print("Sent `{}` to topic `{}`".format(msg, pub_topic))
        else:
            print("Failed to send message to topic {}".format(pub_topic))

    # Function to connect to the MQTT broker as a publisher
    def connect_pub_mqtt():

        def on_connect(client, userdata, flags, rc):
            if rc == 0:
                print("Connected to MQTT Broker!")
            else:
                print("Failed to connect, return code %d\n", rc)

        client = mqtt_client.Client(pub_client_id)
        client.on_connect = on_connect
        client.connect(broker, pub_port)
        return client

    # Function to gather data by publishing a request message
    def gather_data():
        pub_client = connect_pub_mqtt()
        pub_client.loop_start()
        request_data(pub_client)
        pub_client.loop_stop()

    # Function to assign received data to Prometheus metrics
    def assign_data(prom_var, data_name, sensor, jsondata):

        value =  float(jsondata[data_name].replace(',', '.')) # Parse the value
        if  value != -200:  # Exclude no readings
            prom_var.labels(sensor).set(value) # Set the metric value with sensor label

    # Function to process received MQTT messages and update Prometheus metrics
    def promethify_data(msg):

        sensor = msg.topic.split('/')[1] # Extract sensor name from topic
        # Decode and parse the JSON payload
        jsondata_string = msg.payload.decode().replace("'", '"')
        jsondata = json.loads(jsondata_string)

        # Assign each metric to the corresponding Prometheus variable
        assign_data(air_quality_co_gt_gauge, "CO(GT)", sensor, jsondata)
        assign_data(air_quality_pt08s1_co_gauge, "PT08.S1(CO)", sensor, jsondata)
        assign_data(air_quality_nmhc_gt_gauge, "NMHC(GT)", sensor, jsondata)
        assign_data(air_quality_c6h6_gt_gauge, "C6H6(GT)", sensor, jsondata)
        assign_data(air_quality_pt08s2_nmhc_gauge, "PT08.S2(NMHC)", sensor, jsondata)
        assign_data(air_quality_nox_gt_gauge, "NOx(GT)", sensor, jsondata)
        assign_data(air_quality_pt08s3_nox_gauge, "PT08.S3(NOx)", sensor, jsondata)
        assign_data(air_quality_no2_gt_gauge, "NO2(GT)", sensor, jsondata)
```

```
    assign_data(air_quality_pt08s4_no2_gauge, "PT08.S4(NO2)", sensor, jsondata)
    assign_data(air_quality_pt08s5_o3_gauge, "PT08.S5(O3)", sensor, jsondata)
    assign_data(air_quality_t_gauge, "T", sensor, jsondata)
    assign_data(air_quality_rh_gauge, "RH", sensor, jsondata)
    assign_data(air_quality_ah_gauge, "AH", sensor, jsondata)
```

Controller Node Prometheus Metrics Variables

```
from prometheus_client import Gauge

sub_port = 1883 # Port for the subscriber
pub_port = 1883 # Port for the publisher
sub_topic = "sensor-data/#" # Subscription topic for sensor data (wildcard for all
↪  subtopics)
pub_topic = "collect-data" # Topic to publish data collection requests
sub_client_id = 'subscribe-exporter' # Client ID for the MQTT subscriber
pub_client_id = 'publish-exporter' # Client ID for the MQTT publisher
#broker = 'localhost' # uncomment for local testing
broker = 'host.docker.internal' # Docker-specific hostname for connecting to the
↪  broker


# Prometheus metrics definitions
# Each metric is defined as a Prometheus Gauge with a description and a label for
↪  'sensor'

air_quality_co_gt_gauge = Gauge('air_quality_co_gt_gauge', 'True hourly averaged
↪  concentration CO in mg/m^3 (reference analyzer)', ['sensor'])

air_quality_pt08s1_co_gauge = Gauge('air_quality_pt08s1_co_gauge', 'Tin oxide hourly
↪  averaged sensor response (nominally CO targeted)', ['sensor'])

air_quality_nmhc_gt_gauge = Gauge('air_quality_nmhc_gt_gauge', 'True hourly averaged
↪  overall Non Metanic HydroCarbons concentration in microg/m^3 (reference
↪  analyzer)', ['sensor'])

air_quality_c6h6_gt_gauge = Gauge('air_quality_c6h6_gt_gauge', 'True hourly averaged
↪  Benzene concentration in microg/m^3 (reference analyzer)', ['sensor'])

air_quality_pt08s2_nmhc_gauge = Gauge('air_quality_pt08s2_nmhc_gauge', 'Titania hourly
↪  averaged sensor response (nominally NMHC targeted)', ['sensor'])

air_quality_nox_gt_gauge = Gauge('air_quality_nox_gt_gauge', 'True hourly averaged NOx
↪  concentration in ppb (reference analyzer)', ['sensor'])

air_quality_pt08s3_nox_gauge = Gauge('air_quality_pt08s3_nox_gauge', 'Tungsten oxide
↪  hourly averaged sensor response (nominally NOx targeted)', ['sensor'])
```

```
air_quality_no2_gt_gauge = Gauge('air_quality_no2_gt_gauge', 'True hourly averaged NO2
↪  concentration in microg/m^3 (reference analyzer)', ['sensor'])

air_quality_pt08s4_no2_gauge = Gauge('air_quality_pt08s4_no2_gauge', 'Tungsten oxide
↪  hourly averaged sensor response (nominally NO2 targeted)', ['sensor'])

air_quality_pt08s5_o3_gauge = Gauge('air_quality_pt08s5_o3_gauge', 'Indium oxide
↪  hourly averaged sensor response (nominally O3 targeted)', ['sensor'])

air_quality_t_gauge = Gauge('air_quality_t_gauge', 'Temperature in C', ['sensor'])

air_quality_rh_gauge = Gauge('air_quality_rh_gauge', 'Relative Humidity (%)',
↪  ['sensor'])

air_quality_ah_gauge = Gauge('air_quality_ah_gauge', 'Absolute Humidity', ['sensor'])
```

Controller Node Dockerfile

```
FROM python:3.11.0

# Set the working directory inside the container
WORKDIR /srv/exporter

# Add the main application and supporting scripts to the working directory
ADD exporter.py /srv/exporter/
ADD exporter_func.py /srv/exporter/
ADD exporter_var.py /srv/exporter/
ADD requirements.txt /srv/exporter/

# Set execute permissions for all users on the project directory
RUN chmod -R a+x /srv/exporter

# Install the Python dependencies specified in requirements.txt
RUN python3 -m pip install -r requirements.txt

# Expose the necessary ports for the Prometheus metrics endpoint and the MQTT protocol
EXPOSE 8000
EXPOSE 1883

# Set the entrypoint command to run the application using Gunicorn with Uvicorn
↪  workers
ENTRYPOINT [ "gunicorn", "-b", "0.0.0.0:8000", "exporter:app", "-k",
↪  "uvicorn.workers.UvicornWorker" ]
```

Prometheus Configuration File

```
    global:
    scrape_interval:     15s
    evaluation_interval: 15s

    rule_files:
    - prometheus_alerts_rules.yml

    scrape_configs:

    - job_name: prometheus # Collecting data from Prometheus itself
        static_configs:
        - targets: ['127.0.0.1:9090']

    - job_name: sensors # Collecting metrics from the Controller Node
        scrape_interval: 60s
        scrape_timeout: 50s
        static_configs:
        - targets: ['host.docker.internal:8000'] # Using host.docker.internal to direct
        ↪  containerized prometheus to host's 127.0.0.1 (localhost)
```

Docker Compose File

```
services:
    mqtt_broker:
        image: "eclipse-mosquitto:2.0.18"  # Using the Eclipse Mosquitto MQTT broker
        container_name: "thesis-app.mqtt_broker"
        volumes:
        - ./mqtt_broker/mosquitto.conf:/mosquitto/config/mosquitto.conf  # Mount custom
        ↪  Mosquitto configuration
        ports:
        - "1883:1883"  # MQTT default port
        - "9001:9001"  # WebSockets support for MQTT
        restart: unless-stopped  # Automatically restart unless manually stopped

    exporter:
        image: "anastzampetis/iot-exporter"  # Custom IoT data exporter
        container_name: "thesis-app.exporter"
        ports:
        - "8000:8000"  # Expose exporter service on port 8000
        restart: unless-stopped  # Ensure the service restarts if it fails

    prometheus:
        image: "prom/prometheus:v2.46.0"  # Prometheus monitoring service
        container_name: "thesis-app.prometheus"
        volumes:
        - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml  # Custom Prometheus
        ↪  config
```

```
      -
      ↪ ./prometheus/prometheus_alerts_rules.yml:/etc/prometheus/prometheus_alerts_rules.yml
      ↪ # Alert rules (Empty but needed as file)
      - prometheus_data:/prometheus  # Persistent data storage
      ports:
      - "9090:9090"  # Expose Prometheus on port 9090
      command:
      - "--config.file=/etc/prometheus/prometheus.yml"  # Specify config file
      - "--web.enable-lifecycle"  # Allow configuration reload via API
      - "--storage.tsdb.path=/prometheus"  # Set data storage location
      - "--storage.tsdb.retention.time=1y"  # Retention policy. Keep data for 1 year
      restart: unless-stopped

  grafana:
      image: "grafana/grafana:10.2.2"  # Grafana visualization service
      container_name: "thesis-app.grafana"
      ports:
      - "3000:3000"  # Expose Grafana GUI on port 3000
      restart: unless-stopped
      environment:
      - GF_SECURITY_ADMIN_USER=admin  # Admin username for Grafana
      - GF_SECURITY_ADMIN_PASSWORD=grafana  # Admin password for Grafana
      - GF_PATHS_DATA=/var/lib/grafana  # Set data storage path
      volumes:
      - ./grafana:/var/lib/grafana  # Persist Grafana data

  # Sensor services - each sensor must be initialized separately because Docker replicas
  ↪ do not support dynamic variables and container names are not accessible from
  ↪ inside the container. (needed for MQQT client)
  sensor-1:
      image: "anastzampetis/sensor-emul"  # Custom sensor emulator image
      container_name: "thesis-app.sensor-1"
      hostname: "sensor-1"  # Set hostname for usage in the MQTT client
      restart: unless-stopped

  sensor-2:
      image: "anastzampetis/sensor-emul"
      container_name: "thesis-app.sensor-2"
      hostname: "sensor-2"
      restart: unless-stopped

  sensor-3:
      image: "anastzampetis/sensor-emul"
      container_name: "thesis-app.sensor-3"
      hostname: "sensor-3"
      restart: unless-stopped

  sensor-4:
      image: "anastzampetis/sensor-emul"
      container_name: "thesis-app.sensor-4"
```

```
        hostname: "sensor-4"
        restart: unless-stopped

    sensor-5:
        image: "anastzampetis/sensor-emul"
        container_name: "thesis-app.sensor-5"
        hostname: "sensor-5"
        restart: unless-stopped

volumes:
    prometheus_data:  # Persistent volume for Prometheus data storage
```

University of Patras, Polytechnic School
Department of Electrical Engineering & Computer Technology
Anastasios Zampetis of Michael